lib_estat

Biblioteca Estatística em C para Variáveis Aleatórias

Relatório Técnico

Mariana Keiko Ishiy Corrêa
Maria Carolina Sobral Vila Nova de Carvalho Vieira
Eddy Rodrigues Sousa
Luís Guilherme de Oliveira Pelucio

19 de outubro de 2026, Técnicas de Programação

Sumário

1 Introdução **5**
1.1 Objetivo do projeto . . . 5
1.2 Domínio escolhido . . . 5
1.3 Escopo . . . 5
1.4 Organização do relatório . . . 5

2 Organização do Grupo e do Projeto **5**
2.1 Divisão de tarefas . . . 5
2.2 Estrutura de diretórios . . . 6

3 Arquitetura da Biblioteca **6**
3.1 Visão geral dos módulos . . . 6
3.2 Funções públicas . . . 6
3.3 Tratamento de erros . . . 7
3.4 Decisões de projeto . . . 7

4 Descrição dos Módulos **7**
4.1 Esperança dos modelos . . . 7
4.1.1 Fórmulas . . . 8
4.1.2 Assinaturas das funções utilizadas . . . 8
4.1.3 Validações . . . 8
4.1.4 Exemplos numéricos . . . 8
4.2 Variância dos modelos . . . 8
4.2.1 Fórmulas . . . 9
4.2.2 Assinaturas das funções utilizadas . . . 9
4.2.3 Validações . . . 9
4.2.4 Exemplo de implementação . . . 9
4.2.5 Exemplos numéricos . . . 10
4.3 Distribuições de probabilidade . . . 10
4.3.1 Função de probabilidade e função de densidade . . . 10
4.3.2 Fórmulas . . . 10
4.3.3 Cálculo do coeficiente binomial . . . 11
4.3.4 Assinaturas das funções utilizadas . . . 12
4.3.5 Validações . . . 12
4.3.6 Decisões de projeto . . . 12
4.3.7 Exemplo de implementação . . . 12
4.3.8 Exemplos numéricos . . . 13
4.4 Covariância, correlação e regressão . . . 14
4.4.1 Covariância amostral . . . 14
4.4.2 Correlação de Pearson . . . 15
4.4.3 Regressão linear simples . . . 15
4.4.4 Interface pública . . . 15
4.4.5 Tratamento de erros . . . 16
4.4.6 Exemplo numérico . . . 16

5 Testes Automatizados **17**
5.1 Padrão adotado . . . 17
5.2 Estratégia de casos . . . 17
5.3 Cobertura . . . 17
5.3.1 Testes do módulo de variância . . . 18
5.3.2 Testes do módulo de distribuições . . . 18
5.3.3 Testes do módulo de correlação . . . 18
5.4 Como executar . . . 19

6 Depuração com GDB **20**
6.1 Estado inicial (testes passando) . . . 21
6.2 Defeito introduzido . . . 21
6.3 Falha do teste . . . 21
6.4 Sessão de depuração . . . 21
6.5 Correção e verificação . . . 21
6.6 Lições . . . 21
6.7 Inspeção da correlação de Pearson com GDB . . . 21

7 Aplicação Cliente **23**
7.1 Objetivo . . . 23
7.2 Estrutura do programa . . . 23
7.3 Chamadas inválidas de propósito . . . 23
7.4 Saída da aplicação . . . 23
7.5 Versões estática e dinâmica . . . 24

8 Compilação e Distribuição **24**
8.1 Makefile . . . 24
8.2 Biblioteca estática e dinâmica . . . 25
8.3 Compilação incremental . . . 25
8.4 Localização da biblioteca dinâmica . . . 25
8.5 Instalação da biblioteca . . . 25
8.5.1 Verificação da instalação . . . 26

9 Documentação com Doxygen **27**
9.1 Convenções de comentário . . . 27
9.2 Geração . . . 28
9.3 Verificação . . . 28

10 Controle de Versão **29**
10.1 Fluxo de branches . . . 29
10.2 Histórico . . . 30
10.3 Conflitos e como foram resolvidos . . . 31
10.4 Versão `v1.0` . . . 31

11 Dificuldades e Soluções **31**

12 Conclusão **31**

A Código-fonte **32**
A.1 `include/correlacao.h` 32
A.2 `src/correlacao.c` 33
A.3 `tests/test_correlacao.c` 36
A.4 `include/variancia_modelos.h` 39
A.5 `src/variancia_modelos.c` 41
A.6 `tests/test_variancia_modelos.c` 42
A.7 `include/distribuicoes.h` 45
A.8 `src/distribuicoes.c` 46
A.9 `tests/test_distribuicoes.c` 48
A.10 `app/main.c` 54

B Saídas completas **56**

C README **56**

1 Introdução

1.1 Objetivo do projeto

O projeto tem como objetivo fornecer uma biblioteca em C que implemente funções para auxiliar o seu usuário a calcular medidas estatísticas de variáveis aleatórias, como esperança, variância, distribuições de probabilidade e medidas de associação entre duas variáveis aleatórias.

1.2 Domínio escolhido

Variáveis aleatórias, com quatro modelos: Binomial, Geométrica, Uniforme e Poisson, mais medidas de associação entre duas variáveis (covariância, correlação e regressão linear).

1.3 Escopo

Resumo do que a biblioteca entrega: 15 funções públicas em 4 módulos, aplicação cliente, Makefile, testes, documentação Doxygen e versão `v1.0`.

1.4 Organização do relatório

Este relatório descreve o desenvolvimento da biblioteca `lib_estat`. Primeiro são apresentadas a divisão de tarefas, a estrutura de diretórios, a arquitetura da biblioteca, o tratamento de erros e as decisões de projeto. Em seguida, cada um dos quatro módulos é descrito, e são apresentados os testes automatizados, a depuração com GDB e a aplicação cliente. Depois vêm a compilação das bibliotecas estática e dinâmica, a documentação com Doxygen e o controle de versão. O relatório termina com as dificuldades encontradas, a conclusão e os apêndices, que reúnem o código-fonte e as saídas completas.

2 Organização do Grupo e do Projeto

2.1 Divisão de tarefas

Integrante	Módulo	Frente extra
Mariana Keiko Ishiy Corrêa	Esperança dos modelos	Makefile, bibliotecas `.a` e `.so`
Maria Carolina Sobral Vila Nova de Carvalho Vieira	Variância dos modelos	Testes automatizados, aplicação cliente e README
Eddy Rodrigues Sousa	Distribuições de probabilidade	Doxygen e Relatório técnico
Luís Guilherme de Oliveira Pelucio	Covariância, correlação e regressão	Git (branches, merges, tag `v1.0`) e depuração com GDB

Tabela 1: Divisão de tarefas do grupo.

2.2 Estrutura de diretórios

```
projeto2/
|-- include/   (cabecalhos .h: esperanca, variancia_modelos,
|               distribuicoes, correlacao, erros)
|-- src/       (implementacoes .c)
|-- tests/     (um test_*.c por modulo)
|-- app/       (main.c da aplicacao cliente)
|-- lib/       (libestat.a e libestat.so)
|-- build/     (objetos .o)
|-- bin/       (executaveis)
|-- docs/      (Doxygen e evidencias do GDB)
|-- Makefile
‘-- README.md
```

3 Arquitetura da Biblioteca

3.1 Visão geral dos módulos

A construção do código foi feito em etapas, utilizanod cabeçalhos públicos.

```
include/*.h   interfaces publicas (prototipos e comentarios Doxygen
   )
     |
src/*.c       implementacoes das funcoes
     |
build/*.o     objetos (build/pic/*.o para a versao dinamica)
     |
lib/          libestat.a (estatica) e libestat.so (dinamica)
     |
app/main.c, tests/*.c    programas que usam a biblioteca
     |
bin/          executaveis (app, app_dyn, test_*)
```

3.2 Funções públicas

Módulo	Função	Resp.
Esperança	`esperanca_binomial`, `esperanca_geometrica`, `esperanca_uniforme`, `esperanca_poisson`	A
Variância	`variancia_binomial`, `variancia_geometrica`, `variancia_uniforme`, `variancia_poisson`	B
Distribuições	`probabilidade_binomial`, `probabilidade_geometrica`, `probabilidade_poisson`, `densidade_uniforme`	C
Correlação	`covariancia`, `correlacao`, `coeficiente_linear_regressao`	D

Tabela 2: As 15 funções públicas da biblioteca.

3.3 Tratamento de erros

Todas as funções recebem um parâmetro `int *erro`, preenchido com um código definido em `include/erros.h`. Em caso de erro, a função devolve um valor neutro e informa a causa pelo código.

Constante	Valor	Significado
`ESTAT_OK`	0	Sem erro
`ESTAT_ERRO_N_INVALIDO`	1	Tamanho n inválido (por exemplo, $n \leq 0$)
`ESTAT_ERRO_TAMANHOS_DIFERENTES`	2	Vetores de tamanhos diferentes (reservado: nenhuma função o utiliza)
`ESTAT_ERRO_DIVISAO_ZERO`	3	Divisão por zero (por exemplo, $p = 0$ na geométrica)
`ESTAT_ERRO_PARAMETRO_INVALIDO`	4	Parâmetro fora do domínio (por exemplo, $p \notin [0, 1]$, $a \geq b$ ou vetor `NULL`)

Tabela 3: Códigos de erro da biblioteca.

O ponteiro `erro` não pode ser `NULL`, com uma exceção: as funções do módulo de correlação aceitam `erro == NULL`, caso em que nenhum código de erro é armazenado. Nesse módulo, um vetor `NULL` resulta em `ESTAT_ERRO_PARAMETRO_INVALIDO`.

3.4 Decisões de projeto

- Erro via parâmetro `int *erro` (e não via `errno` ou `exit`), para que a biblioteca nunca encerre o programa do usuário, uma vez que a biblioteca é uma ferramenta de apoio, e não um programa completo.
- Cabeçalhos independentes, cada um incluindo apenas `erros.h`.
- Convenção da distribuição geométrica: $k = 0, 1, 2, \ldots$ (número de falhas até o primeiro sucesso). Uma vez que há uma variação em cada representação da distribuição geométrica, decidimos adotar a convenção de k como o número de falhas até o primeiro sucesso, que é a forma mais intuitiva, no estudo de variáveis aleatórias.
- Tratamento de $p = 0$ na geométrica: tanto `esperanca_geometrica` quanto `variancia_geometrica` devolvem `ESTAT_ERRO_DIVISAO_ZERO`, pois as fórmulas $(1 - p)/p$ e $(1 - p)/p^2$ dividem por p. Valores de p fora de $[0, 1]$ resultam em `ESTAT_ERRO_PARAMETRO_INVALIDO`.

4 Descrição dos Módulos

4.1 Esperança dos modelos

Este módulo calcula a esperança dos quatro modelos (geométrico, uniforme, binomial e Poisson). Os protótipos estão em `include/esperanca.h` e a implementação em `src/esperanca.c`.

Modelo	Parâmetros	Esperança
Binomial	n, p	$E[X] = n\,p$
Geométrica	p	$E[X] = \dfrac{1-p}{p}$
Uniforme	a, b	$E[X] = \dfrac{a+b}{2}$
Poisson	λ	$E[X] = \lambda$

Tabela 4: Esperança dos modelos implementados.

4.1.1 Fórmulas

4.1.2 Assinaturas das funções utilizadas

```c
double esperanca_binomial(int n, double p, int *erro);
double esperanca_geometrica(double p, int *erro);
double esperanca_uniforme(double a, double b, int *erro);
double esperanca_poisson(double lambda, int *erro);
```

4.1.3 Validações

Função	Condição	Código de erro
`esperanca_binomial`	$n \leq 0$	`ESTAT_ERRO_N_INVALIDO`
	$p < 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`esperanca_geometrica`	$p < 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
	$p = 0$	`ESTAT_ERRO_DIVISAO_ZERO`
`esperanca_uniforme`	$a \geq b$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`esperanca_poisson`	$\lambda \leq 0$	`ESTAT_ERRO_PARAMETRO_INVALIDO`

Tabela 5: Validações de entrada do módulo de esperança.

4.1.4 Exemplos numéricos

Chamada	Resultado
`esperanca_binomial(10, 0.5)`	5
`esperanca_geometrica(0.5)`	1
`esperanca_uniforme(5, 10)`	7,5
`esperanca_poisson(10)`	10

Tabela 6: Valores conferidos pelos testes e pela aplicação cliente.

4.2 Variância dos modelos

Este módulo calcula a variância dos quatro modelos de variáveis aleatórias. Os protótipos estão em `include/variancia_modelos.h` e a implementação em `src/variancia_modelos.c`.

4.2.1 Fórmulas

Modelo	Parâmetros	Variância
Binomial	n, p	$\mathrm{Var}(X) = n\,p\,(1-p)$
Geométrica	p	$\mathrm{Var}(X) = \dfrac{1-p}{p^2}$
Uniforme	a, b	$\mathrm{Var}(X) = \dfrac{(b-a)^2}{12}$
Poisson	λ	$\mathrm{Var}(X) = \lambda$

Tabela 7: Variância dos modelos implementados.

Na geométrica foi usada a fórmula fechada $(1-p)/p^2$, e não o cálculo a partir de $E[X^2]-E[X]^2$, porque a primeira é mais eficiente e não há risco de subtração de números próximos que causem perda de precisão.

4.2.2 Assinaturas das funções utilizadas

```c
double variancia_binomial(int n, double p, int *erro);
double variancia_geometrica(double p, int *erro);
double variancia_uniforme(double a, double b, int *erro);
double variancia_poisson(double lambda, int *erro);
```

4.2.3 Validações

Função	Condição	Código de erro
`variancia_binomial`	$n \leq 0$	`ESTAT_ERRO_N_INVALIDO`
	$p < 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`variancia_geometrica`	$p < 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
	$p = 0$	`ESTAT_ERRO_DIVISAO_ZERO`
`variancia_uniforme`	$a \geq b$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`variancia_poisson`	$\lambda \leq 0$	`ESTAT_ERRO_PARAMETRO_INVALIDO`

Tabela 8: Validações de entrada do módulo de variância.

4.2.4 Exemplo de implementação

```c
double variancia_binomial(int n, double p, int *erro){
    if (n <= 0) {
        *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }
    if (p < 0 || p > 1) {
        *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }
```

```
10
11      *erro = ESTAT_OK;
12      return n*p*(1-p);
13
14
15  }
```

4.2.5 Exemplos numéricos

Chamada	Resultado
`variancia_binomial(10, 0.5)`	2,5
`variancia_geometrica(0.5)`	2
`variancia_uniforme(0, 10)`	8,3333
`variancia_poisson(4)`	4

Tabela 9: Valores conferidos manualmente e pela aplicação cliente.

4.3 Distribuições de probabilidade

Este módulo calcula a função de probabilidade dos modelos discretos (Binomial, Geométrica e Poisson) e a função de densidade do modelo contínuo Uniforme. Os protótipos estão em `include/distribuicoes.h` e a implementação em `src/distribuicoes.c`.

4.3.1 Função de probabilidade e função de densidade

Os três modelos discretos retornam uma probabilidade pontual, $P(X = k)$. Já o modelo Uniforme é contínuo, e por isso `densidade_uniforme` retorna a densidade $f(x)$, que **não** é uma probabilidade pontual: em uma distribuição contínua, $P(X = x) = 0$ para qualquer x, e a probabilidade só faz sentido para intervalos, obtida pela integral da densidade. Por isso o valor de $f(x)$ pode, inclusive, ser maior que 1 (por exemplo, $f(x) = 2$ em $[0, 0{,}5]$).

4.3.2 Fórmulas

Modelo	Parâmetros	Função
Binomial	n, p	$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$
Geométrica	p	$P(X = k) = p\,(1-p)^k$
Poisson	λ	$P(X = k) = \dfrac{e^{-\lambda}\,\lambda^k}{k!}$
Uniforme	a, b	$f(x) = \begin{cases} \dfrac{1}{b-a} & \text{se } a \le x \le b \\ 0 & \text{caso contrário} \end{cases}$

Tabela 10: Funções calculadas pelo módulo de distribuições.

Definição da geométrica. Foi adotada a definição em que X é o número de *fracassos antes do primeiro sucesso*, com $k = 0, 1, 2, \ldots$ Na definição alternativa (número de tentativas até o primeiro sucesso, $k = 1, 2, \ldots$) a função seria $p\,(1-p)^{k-1}$. Essa escolha é a mesma nos módulos de esperança e variância (ver Seção 11), pois a esperança muda entre as duas definições ($(1-p)/p$ contra $1/p$).

Cálculo numérico. Para evitar estouro (*overflow*), dois cálculos não usam o fatorial diretamente:

- o coeficiente binomial $\binom{n}{k}$ é calculado de forma iterativa, pela função auxiliar `binomio`, sem calcular $n!$ (detalhada na Seção 4.3.3);
- na Poisson, o cálculo é feito em escala logarítmica: $P(X = k) = \exp\big(-\lambda + k \ln \lambda - \ln(k!)\big)$, onde $\ln(k!)$ é obtido por `lgamma(k + 1)`.

4.3.3 Cálculo do coeficiente binomial

O coeficiente $\binom{n}{k} = \dfrac{n!}{k!\,(n-k)!}$ não é calculado pela definição, porque $n!$ ultrapassa o limite de um `double` já para n próximo de 170, mesmo quando o coeficiente em si é pequeno. A função auxiliar `binomio` (declarada `static`, portanto interna ao módulo) usa o produto

$$\binom{n}{k} = \prod_{i=1}^{k} \frac{n-k+i}{i}.$$

```
1  static double binomio(double n, double k){
2    if (k > n - k) k = n - k;
3    double c = 1.0;
4    for (int i = 1; i <= k; i++) {
5      c = c * (n - k + i) / i;
6    }
7    return c;
8  }
```

O funcionamento tem três ideias:

1. **Simetria.** Como $\binom{n}{k} = \binom{n}{n-k}$, a linha `if (k > n - k) k = n - k;` troca k pelo menor dos dois valores, o que reduz o número de iterações para $\min(k, n-k)$.
2. **Invariante do laço.** Ao fim da iteração i, a variável `c` vale exatamente $\binom{n-k+i}{i}$, pois $\binom{n-k+i}{i} = \binom{n-k+i-1}{i-1} \cdot \dfrac{n-k+i}{i}$. Na última iteração ($i = k$), `c` vale $\binom{n}{k}$.
3. **Multiplicar antes de dividir.** Em `c * (n - k + i) / i` a multiplicação é feita primeiro, e a divisão por i é sempre exata, já que o resultado é um coeficiente binomial inteiro. Isso evita acumular erros de arredondamento.

Exemplo: $\binom{10}{3}$. Como $3 \le 10-3$, o valor de k não muda, e o laço executa três iterações:

i	Conta	`c`
1	$1 \cdot 8/1$	$8 = \binom{8}{1}$
2	$8 \cdot 9/2$	$36 = \binom{9}{2}$
3	$36 \cdot 10/3$	$120 = \binom{10}{3}$

Nenhum fatorial é calculado: o maior valor intermediário é o próprio resultado.

Limitação. O coeficiente em si ainda é um `double`. Para n da ordem de mil, $\binom{n}{k}$ pode ultrapassar o maior valor representável, e p^k pode ficar abaixo do menor, de modo que o produto final deixa de ser confiável para n muito grande. Isso está fora do escopo dos testes do projeto.

4.3.4 Assinaturas das funções utilizadas

```c
double probabilidade_binomial(int k, int n, double p, int *erro);
double probabilidade_geometrica(int k, double p, int *erro);
double probabilidade_poisson(int k, double lambda, int *erro);
double densidade_uniforme(double x, double a, double b, int *erro);
```

4.3.5 Validações

Função	Condição	Código de erro
`probabilidade_binomial`	$n \leq 0$ ou $k < 0$	`ESTAT_ERRO_N_INVALIDO`
	$p < 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`probabilidade_geometrica`	$k < 0$	`ESTAT_ERRO_N_INVALIDO`
	$p \leq 0$ ou $p > 1$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`probabilidade_poisson`	$k < 0$	`ESTAT_ERRO_N_INVALIDO`
	$\lambda \leq 0$	`ESTAT_ERRO_PARAMETRO_INVALIDO`
`densidade_uniforme`	$a \geq b$	`ESTAT_ERRO_PARAMETRO_INVALIDO`

Tabela 11: Validações de entrada do módulo de distribuições.

4.3.6 Decisões de projeto

Dois casos retornam 0,0 **sem** sinalizar erro (`ESTAT_OK`), porque são matematicamente corretos:

- `probabilidade_binomial` com $k > n$: não é possível obter mais sucessos que tentativas, logo $P(X = k) = 0$;
- `densidade_uniforme` com x fora de $[a, b]$: a densidade é nula fora do suporte da distribuição.

Essa decisão para a densidade uniforme era uma pendência registrada na arquitetura, e foi resolvida seguindo a sugestão do documento. Além disso, em `probabilidade_geometrica` o valor $p = 0$ é tratado como parâmetro fora do domínio ($0 < p \leq 1$) e devolve `ESTAT_ERRO_PARAMETRO_INVALIDO`, pois a fórmula $p\,(1-p)^k$ não envolve divisão por p.

4.3.7 Exemplo de implementação

```c
double probabilidade_poisson(int k, double lambda, int *erro){
    if (k < 0) {
        *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }
    if (lambda <= 0) {
        *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }

    *erro = ESTAT_OK;
    /* escala logaritmica evita overflow do fatorial */
    return exp(-lambda + k * log(lambda) - lgamma(k + 1.0));
}
```

4.3.8 Exemplos numéricos

Os exemplos abaixo são os casos do teste automatizado `test_distribuicoes.c`, que contém 25 testes com `assert`. Ele foi compilado junto com `distribuicoes.c` e todos os testes passaram. Nas tabelas, o prefixo `test_probabilidade_` dos nomes dos testes foi omitido por brevidade.

Teste	Chamada	Resultado
`binomial_valida`	`probabilidade_binomial(1, 3, 0.5)`	0,375
`geometrica`	`probabilidade_geometrica(2, 0.5)`	0,125
`poisson`	`probabilidade_poisson(0, 2.0)`	$e^{-2} \approx 0{,}1353$
`densidade_uniforme`	`densidade_uniforme(1.0, 0.0, 4.0)`	0,25
`densidade_uniforme`	`densidade_uniforme(5.0, 0.0, 4.0)`	0 (x fora de $[a, b]$)
`binomial_k_maior_que_n`	`probabilidade_binomial(4, 3, 0.5)`	0 ($k > n$, sem erro)
`binomial_coeficiente`	`probabilidade_binomial(3, 10, 0.5)`	0,1172
`binomial_coeficiente`	`probabilidade_binomial(7, 10, 0.5)`	0,1172 ($k > n - k$)
`binomial_p_limites`	`probabilidade_binomial(4, 4, 1.0)`	1 ($p = 1$)
`binomial_n_grande`	`probabilidade_binomial(100, 200, 0.5)`	0,0563
`geometrica_p_um`	`probabilidade_geometrica(0, 1.0)`	1 ($p = 1$)
`poisson_k_positivo`	`probabilidade_poisson(2, 3.0)`	0,2240
`poisson_k_grande`	`probabilidade_poisson(200, 200.0)`	0,0282
`densidade_uniforme_extremos`	`densidade_uniforme(4.0, 0.0, 4.0)`	0,25 ($x = b$)

Tabela 12: Casos válidos do teste automatizado, todos com `ESTAT_OK`.

Casos válidos. Conferência manual dos três primeiros: $\binom{3}{1} 0{,}5^1\, 0{,}5^2 = 3 \cdot 0{,}125 = 0{,}375$; $0{,}5 \cdot 0{,}5^2 = 0{,}125$; e, para $k = 0$, a Poisson se reduz a $e^{-\lambda}$. A densidade uniforme em $[0, 4]$ vale $1/(4 - 0) = 0{,}25$.

Casos de erro. Em todos eles o teste também confere que o valor retornado é 0,0. Nos códigos de erro, o prefixo `ESTAT_ERRO_` foi omitido.

Valores grandes. Os testes `binomial_n_grande` e `poisson_k_grande` verificam o cálculo em casos em que a conta direta estouraria um `double`: na binomial, 200! (da ordem de

Teste	Chamada	Código de erro
`binomial_n_invalido`	`probabilidade_binomial(1, -1, 1.0)`	`N_INVALIDO`
`binomial_n_zero`	`probabilidade_binomial(0, 0, 0.5)`	`N_INVALIDO`
`binomial_k_menor_que_zero`	`probabilidade_binomial(-1, 3, 0.5)`	`N_INVALIDO`
`binomial_p_invalida`	`probabilidade_binomial(1, 3, -1.0)`	`PARAMETRO_INVALIDO`
`binomial_p_maior_que_um`	`probabilidade_binomial(1, 3, 1.5)`	`PARAMETRO_INVALIDO`
`geometrica_k_invalido`	`probabilidade_geometrica(-1, 0.5)`	`N_INVALIDO`
`geometrica_p_invalida`	`probabilidade_geometrica(2, -0.5)`	`PARAMETRO_INVALIDO`
`geometrica_p_maior_que_um`	`probabilidade_geometrica(2, 1.5)`	`PARAMETRO_INVALIDO`
`geometrica`	`probabilidade_geometrica(0, 0.0)`	`PARAMETRO_INVALIDO`
`poisson_k_invalido`	`probabilidade_poisson(-1, 2.0)`	`N_INVALIDO`
`poisson_lambda_invalido`	`probabilidade_poisson(2, -1)`	`PARAMETRO_INVALIDO`
`poisson_lambda_zero`	`probabilidade_poisson(2, 0.0)`	`PARAMETRO_INVALIDO`
`densidade_uniforme_a_invalido`	`densidade_uniforme(1, 5, 3)`	`PARAMETRO_INVALIDO`
`densidade_uniforme_a_igual_b`	`densidade_uniforme(2.0, 2.0, 2.0)`	`PARAMETRO_INVALIDO`

Tabela 13: Casos de erro do teste automatizado.

10^{374}); na Poisson com $k = 200$ e $\lambda = 200$, tanto 200! quanto λ^k (da ordem de 10^{460}) ultrapassam o maior valor representável, cerca de $1{,}8 \times 10^{308}$. O teste `binomial_coeficiente` também cobre o caso $k > n-k$, em que o `binomio` troca k por $n-k$. Todas as comparações com números em ponto flutuante usam a função `aproximadamente` da Seção 5.1.

4.4 Covariância, correlação e regressão

O módulo de correlação foi desenvolvido para analisar a relação linear entre duas variáveis a partir de amostras de dados observados. Diferentemente dos módulos de esperança, variância e distribuições, que utilizam parâmetros de modelos probabilísticos teóricos, este módulo recebe dois vetores de valores reais e realiza os cálculos diretamente sobre as observações.

A implementação está localizada em `src/correlacao.c`, com interface pública definida em `include/correlacao.h`. Foram implementadas três funções: covariância amostral, correlação de Pearson e coeficiente angular da regressão linear simples.

4.4.1 Covariância amostral

A covariância mede como duas variáveis variam conjuntamente. Valores positivos indicam que as variáveis tendem a crescer ou diminuir juntas, enquanto valores negativos indicam uma tendência de variação em sentidos opostos.

A fórmula utilizada é:

$$S_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}), \tag{1}$$

em que $\bar{x}$ e $\bar{y}$ representam as médias das amostras e n é a quantidade de observações.

A implementação percorre inicialmente os vetores para calcular as médias. Em seguida, realiza uma segunda passagem para somar os produtos dos desvios em relação às médias. O resultado é dividido por $n - 1$, correspondendo à covariância amostral.

4.4.2 Correlação de Pearson

O coeficiente de correlação de Pearson mede a intensidade e a direção da relação linear entre duas variáveis.

Seu valor varia entre -1 e 1:

- $r = 1$: correlação linear positiva perfeita;
- $r = -1$: correlação linear negativa perfeita;
- $r = 0$: ausência de correlação linear.

O cálculo é realizado pela fórmula:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}. \tag{2}$$

A implementação calcula as médias das duas amostras e, posteriormente, a soma dos produtos dos desvios e as somas dos quadrados dos desvios de cada variável.

Caso uma das amostras apresente variância nula, a correlação não pode ser calculada, pois o denominador da expressão seria zero. Nessa situação, a função retorna `ESTAT_ERRO_DIVISAO_ZERO`.

4.4.3 Regressão linear simples

A terceira função calcula o coeficiente angular da reta de regressão linear simples pelo método dos mínimos quadrados.

A reta pode ser representada por:

$$y = ax + b, \tag{3}$$

em que a representa o coeficiente angular, responsável pela inclinação da reta, e b representa o coeficiente linear, ou intercepto.

Conforme a convenção definida pelo grupo, a função `coeficiente_linear_regressao` retorna o coeficiente angular a, calculado por:

$$a = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}. \tag{4}$$

Nesse cálculo, a variável X é considerada independente e Y é a variável dependente.

Quando todos os valores de X são iguais, o denominador é zero e não é possível determinar um coeficiente angular único. Nesse caso, a função sinaliza `ESTAT_ERRO_DIVISAO_ZERO`.

4.4.4 Interface pública

As três funções possuem as seguintes assinaturas:

```
1 double covariancia(
2     double amostra_x[],
3     double amostra_y[],
4     int n,
5     int *erro
6 );
7
```

```c
double correlacao(
    double amostra_x[],
    double amostra_y[],
    int n,
    int *erro
);

double coeficiente_linear_regressao(
    double amostra_x[],
    double amostra_y[],
    int n,
    int *erro
);
```

Os parâmetros `amostra_x` e `amostra_y` representam os vetores de observações. O parâmetro `n` indica a quantidade de elementos considerada em cada vetor, enquanto `erro` permite informar o resultado da validação dos parâmetros.

4.4.5 Tratamento de erros

O módulo utiliza os códigos padronizados definidos em `include/erros.h`.

As principais validações implementadas são:

- Para $n \leq 1$, é retornado `ESTAT_ERRO_N_INVALIDO`.
- Se algum vetor recebido for `NULL`, é retornado `ESTAT_ERRO_PARAMETRO_INVALIDO`.
- Na correlação de Pearson, variância nula em qualquer uma das amostras resulta em `ESTAT_ERRO_DIVISAO_ZERO`.
- Na regressão linear, variância nula da amostra X resulta em `ESTAT_ERRO_DIVISAO_ZERO`.

Em caso de erro, as funções retornam 0,0 e registram o código correspondente por meio do ponteiro `erro`, quando ele não é `NULL`. Em operações bem-sucedidas, esse código é `ESTAT_OK`.

Como o valor 0,0 também pode representar um resultado matemático válido, é necessário consultar o código de erro para distinguir um resultado legítimo de uma falha.

A interface utiliza apenas um parâmetro `n` para os dois vetores. Portanto, cabe ao programa cliente garantir que ambos contenham pelo menos n elementos. A função não consegue identificar automaticamente vetores de tamanhos diferentes, pois a linguagem C não disponibiliza seus comprimentos por meio desses parâmetros.

4.4.6 Exemplo numérico

Para demonstrar o funcionamento das três funções, foram utilizadas as seguintes amostras:

$$X = \{1, 2, 3\}, \qquad Y = \{3, 5, 7\}. \tag{5}$$

As médias são $\bar{x} = 2$ e $\bar{y} = 5$.

Aplicando as fórmulas anteriores, obtemos:

$$S_{xy} = \frac{4}{3-1} = 2, \tag{6}$$

$$r = \frac{4}{\sqrt{2}\sqrt{8}} = 1, \tag{7}$$

$$a = \frac{4}{2} = 2. \tag{8}$$

Os resultados correspondem, respectivamente, à covariância amostral, à correlação positiva perfeita e ao coeficiente angular da reta $y = 2x + 1$.

Esses valores foram utilizados na verificação inicial do módulo e posteriormente complementados pelos testes automatizados, incluindo casos válidos, limites e entradas inválidas.

5 Testes Automatizados

5.1 Padrão adotado

Os testes usam `assert` da biblioteca padrão, com uma função auxiliar de comparação com tolerância para números em ponto flutuante:

```
1 int aproximadamente(double x, double y) {
2     return fabs(y - x) < 0.000001;
3 }
```

Escolheu-se `assert` por ser simples, confiável e amplamente usado, e não criar macros próprias que poderiam confundir o leitor. Também utilizou-se a função `aproximadamente` para comparar os números double e evitar problemas de arredondamento que podem ocorrer ao longo do código.

Cada módulo tem um arquivo `tests/test_<modulo>.c`. Quando um teste passa, o programa imprime uma mensagem no terminal (no módulo de correlação, uma por grupo de testes). Quando um teste falha, o `assert` aborta o programa e o `make` interrompe a execução.

5.2 Estratégia de casos

Cada função é testada com (1) caso normal com valor conhecido, (2) cada condição de erro, e (3) valores limite ($p = 0$, $p = 1$, $n = 0$ etc.).

5.3 Cobertura

Nesta tabela, a coluna de testes representa os cenários verificados, que podem estar agrupados em uma mesma função de teste. No módulo de correlação, os 16 cenários são organizados em três funções auxiliares.

Módulo	Funções	Testes
Esperança	4	10
Variância	4	15
Distribuições	4	25
Correlação	3	16
Total	15	71

Tabela 14: Cobertura dos testes por módulo.

Função	Nº de testes	Casos cobertos
Binomial	6	normal, n inválido, $n = 0$, p inválido, $p = 0$, $p = 1$
Geométrica	4	normal, p inválido, divisão por zero, $p = 1$
Uniforme	3	normal, $a \geq b$, a negativo
Poisson	2	normal, λ inválido

Tabela 15: Testes de `variancia_modelos`.

5.3.1 Testes do módulo de variância

5.3.2 Testes do módulo de distribuições

5.3.3 Testes do módulo de correlação

Os testes do módulo estão implementados no arquivo `tests/test_correlacao.c`. Foram elaborados 16 cenários de teste, organizados em três funções auxiliares: `testar_covariancia`, `testar_correlacao` e `testar_regressao`.

Os testes verificam tanto a exatidão dos resultados matemáticos quanto o tratamento de entradas inválidas e situações em que os cálculos não estão definidos.

Para os casos válidos, foram utilizadas amostras com resultados matemáticos conhecidos, como $X = \{1, 2, 3\}$, $Y = \{3, 5, 7\}$ e $Y' = \{7, 5, 3\}$.

A comparação entre resultados esperados e obtidos utiliza uma tolerância numérica de 10^{-9}, evitando falhas causadas por pequenas diferenças de representação em ponto flutuante.

```
static void verificar_resultado(
    double obtido, double esperado
) {
    assert(fabs(obtido - esperado) < 1e-9);
}
```

Além do valor numérico retornado, cada cenário verifica o código de erro produzido pela função. Nos casos válidos, é esperado `ESTAT_OK`. Nos casos inválidos, são verificados os códigos correspondentes a tamanho inválido, parâmetro inválido ou divisão por zero.

Um caso importante é a regressão com valores constantes na variável Y. Nessa situação, o coeficiente angular é igual a zero e o resultado é válido, desde que haja variação na variável X.

Por outro lado, quando X é constante, a regressão não possui coeficiente angular único e deve sinalizar `ESTAT_ERRO_DIVISAO_ZERO`.

Função	Nº de testes	Casos cobertos
Binomial	10	normal, n inválido, $n = 0$, $k < 0$, $k > n$, $p < 0$, $p > 1$, $p = 0$ e $p = 1$, coeficiente com $k > n - k$, n grande
Geométrica	5	normal (com $p = 0$ inválido), k inválido, $p < 0$, $p > 1$, $p = 1$
Poisson	6	normal ($k = 0$), k inválido, $\lambda < 0$, $\lambda = 0$, $k > 0$, k grande
Uniforme	4	normal (dentro e fora de $[a, b]$), $a > b$, $a = b$, extremos $x = a$ e $x = b$

Tabela 16: Testes de `distribuicoes`.

Função	Casos	Situações verificadas
Covariância	4	Covariância positiva (2), negativa (-2), tamanho inválido ($n = 1$) e vetor nulo.
Correlação	6	Correlação positiva perfeita (1), negativa perfeita (-1), variância zero em X, variância zero em Y, tamanho inválido ($n = 0$) e vetor nulo.
Regressão linear	6	Coeficiente angular positivo (2), negativo (-2), coeficiente angular zero válido, variância zero em X, tamanho inválido ($n = 1$) e vetor nulo.
Total	16	

Tabela 17: Casos de teste do módulo de correlação.

Os testes utilizam `assert` para interromper a execução quando uma verificação falha. Ao concluir cada grupo de verificações, o programa imprime uma mensagem de sucesso no terminal.

5.4 Como executar

```
make test
```

A saída abaixo foi obtida com todos os módulos passando (arquivo `saida_test.txt`, gerado por `make -s test`):

```
== bin/test_correlacao
testes de covariancia: OK
testes de correlacao: OK
testes de regressao: OK
todos os testes do modulo passaram!
== bin/test_distribuicoes
[passou] test_probabilidade_binomial_n_invalido
[passou] test_probabilidade_binomial_p_invalida
[passou] test_probabilidade_binomial_valida
[passou] test_probabilidade_binomial_k_menor_que_zero
[passou] test_probabilidade_binomial_k_maior_que_n
[passou] test_probabilidade_binomial_n_zero
[passou] test_probabilidade_geometrica_p_invalida
```

```
[passou] test_probabilidade_geometrica_k_invalido
[passou] test_probabilidade_geometrica
[passou] test_probabilidade_poisson_lambda_invalido
[passou] test_probabilidade_poisson_k_invalido
[passou] test_probabilidade_poisson
[passou] test_densidade_uniforme
[passou] test_densidade_uniforme_a_invalido
[passou] test_densidade_uniforme_a_igual_b
[passou] test_probabilidade_binomial_coeficiente
[passou] test_probabilidade_binomial_p_maior_que_um
[passou] test_probabilidade_binomial_p_limites
[passou] test_probabilidade_binomial_n_grande
[passou] test_probabilidade_geometrica_p_maior_que_um
[passou] test_probabilidade_geometrica_p_um
[passou] test_probabilidade_poisson_k_positivo
[passou] test_probabilidade_poisson_lambda_zero
[passou] test_probabilidade_poisson_k_grande
[passou] test_densidade_uniforme_extremos
== bin/test_esperanca
Passou em test_esperanca_binomial_n_invalido.
Passou em test_esperanca_binomial_parametro_invalido.
Passou em test_esperanca_binomial_valido.
Passou em test_esperanca_geometrica_parametro_invalido.
Passou em test_esperanca_geometrica_valido.
Passou em test_esperanca_uniforme_parametro_invalido
Passou em test_esperanca_uniforme_valido.
Passou em test_esperanca_poisson_parametro_invalido
Passou em test_esperanca_poisson_valido.
Todos os testes obtiveram sucesso!.
== bin/test_variancia_modelos
passou no teste_variancia_binomial_erro_n_invalido
passou no teste_variancia_binomial_erro_p_invalido
passou no teste_variancia_binomial_normal
passou no teste_variancia_binomial_p_igual_1
passou no teste_variancia_binomial_n_igual_0
passou no teste_variancia_binomial_p_igual_0
passou no teste_variancia_geo_erro_div_por_zero
passou no teste_variancia_geo_erro_p_invalido
passou no teste_variancia_geo_normal
passou no teste_variancia_geo_p_igual_1
passou no teste_variancia_poisson_lambda_invalido
passou no teste_variancia_poisson_normal
passou no teste_variancia_uniforme_erro_a_maior_ou_igual_b
passou no teste_variancia_uniforme_normal
passou no teste_variancia_uniforme_a_negativo
```

6 Depuração com GDB

Cenário: introduzir um defeito proposital, observar o teste falhar, localizar a causa com o depurador e corrigir.

6.1 Estado inicial (testes passando)

6.2 Defeito introduzido

Troca do sinal em `variancia_binomial`:

```
return n * p * (1 + p);   /* defeito: o correto e (1 - p) */
```

6.3 Falha do teste

Para $n = 10$ e $p = 0{,}5$ o valor esperado é 2,5, mas o código devolve 7,5.

6.4 Sessão de depuração

Comando	Objetivo
`gdb ./bin/test_variancia_modelos`	abrir o executável compilado com `-g`
`break variancia_binomial`	parar na função suspeita
`run`	executar até o ponto de parada
`info args`	ver n e p recebidos
`next`	avançar uma linha
`print n*p*(1-p)`	valor da fórmula correta
`print n*p*(1+p)`	valor da fórmula do código
`bt`	pilha de chamadas

Tabela 18: Comandos usados na depuração.

6.5 Correção e verificação

6.6 Lições

6.7 Inspeção da correlação de Pearson com GDB

Além do cenário de depuração descrito anteriormente, foi realizada uma sessão de inspeção da função `correlacao`, implementada no módulo `src/correlacao.c`.

O objetivo foi acompanhar os cálculos intermediários da correlação de Pearson, verificar os valores das variáveis locais e confirmar a correção do resultado retornado.

Para permitir a inspeção das variáveis, o executável de testes foi compilado com informações de depuração (`-g`) e sem otimizações (`-O0`):

```
gcc -g -O0 -Wall -Wextra -std=c11 \
    -Iinclude \
    tests/test_correlacao.c \
    src/correlacao.c \
    -o /tmp/test_correlacao_gdb -lm

gdb /tmp/test_correlacao_gdb
```

Dentro do GDB, foi criado um ponto de parada no início da função:

```
break correlacao
run
```

A execução foi interrompida quando o programa entrou na função. Nesse momento, foram inspecionados o tamanho da amostra e os primeiros elementos dos vetores:

```
print n
print amostra_x[0]
print amostra_y[0]
```

Os valores observados foram $n = 3$, $x_1 = 1$ e $y_1 = 3$, correspondentes às amostras de teste $X = \{1, 2, 3\}$ e $Y = \{3, 5, 7\}$.

Em seguida, foram utilizados pontos de parada temporários (`tbreak`) e o comando `continue` para avançar até as etapas do cálculo das médias, das somas e do denominador da correlação.

Variável inspecionada	Valor observado
`n`	3
`soma_x`	6
`soma_y`	15
`media_x`	2
`media_y`	5
`soma_produtos`	4
`soma_quadrados_x`	2
`soma_quadrados_y`	8

Tabela 19: Valores intermediários inspecionados com GDB na função de correlação.

Os valores encontrados são consistentes com a fórmula da correlação de Pearson. Para essas amostras, o numerador é igual a 4 e o denominador matemático também é igual a 4, produzindo uma correlação igual a 1.

Entretanto, durante a inspeção com GDB, foi observado que a variável `denominador` possuía o valor aproximado de 4,0000000000000009. O resultado retornado pela função foi aproximadamente 0,99999999999999978.

Essa pequena diferença em relação aos valores matemáticos exatos ocorre devido às limitações da representação de números em ponto flutuante, utilizada pelo tipo `double` em C.

Por esse motivo, os testes automatizados do módulo não utilizam igualdade exata entre valores reais. Em vez disso, empregam uma tolerância de 10^{-9}:

```
assert(fabs(obtido - esperado) < 1e-9);
```

Essa estratégia permite verificar corretamente resultados numéricos sujeitos a pequenas diferenças de arredondamento.

Ao final da inspeção, o comando `finish` permitiu confirmar o retorno da função. A execução posterior dos testes terminou normalmente, sem falhas.

Diferentemente do cenário de defeito proposital apresentado anteriormente, essa sessão teve caráter de verificação e compreensão do funcionamento interno da implementação. Não foi necessário corrigir um defeito na função de correlação.

A experiência demonstrou a utilidade do GDB para acompanhar variáveis intermediárias, localizar etapas específicas da execução e compreender diferenças entre resultados matemáticos exatos e resultados representados em ponto flutuante.

7 Aplicação Cliente

7.1 Objetivo

Mostrar o uso da biblioteca por um programa externo, que inclui apenas os cabeçalhos públicos e nunca os `.c`.

7.2 Estrutura do programa

Funções `demo_esperanca`, `demo_variancia`, `demo_distribuicoes` e `demo_correlacao`, mais as auxiliares `mensagem_erro` e `imprime_res`, que traduzem o código de erro em texto.

7.3 Chamadas inválidas de propósito

Cada demonstração inclui chamadas com argumentos inválidos para mostrar o tratamento de erros (por exemplo, `variancia_binomial(10, 1.5)`).

7.4 Saída da aplicação

A execução de `./bin/app` produz a saída abaixo (arquivo `saida_app.txt`):

```
=====Esperança dos modelos=====
Esperança Binomial(10,0.5): 5.000000
Esperança Geometrica(0.5): 1.000000
Esperança de Poisson(10): 10.000000
Esperança Uniforme(5,10): 7.500000
Esperança Binomial(0, 0.5) [invalido]: N invalido
Esperança Uniforme(10, 5) [invalido]: Parametro invalido
=====Variância dos modelos=====
Variancia Binomial(10, 0.5): 2.500000
Variancia Geometrica(0.5): 2.000000
Variancia Uniforme(0, 10): 8.333333
Variancia Poisson(4): 4.000000
Variancia Binomial(10, 1.5) [invalido]: Parametro invalido
=====Distribuições=====
P Binomial(k=5, n=10, p=0.5): 0.246094
P Geometrica(k=3, p=0.5): 0.062500
P Poisson(k=2, lambda=4): 0.146525
Densidade Uniforme(x=5, a=0, b=10): 0.100000
P Binomial(k=5, n=10, p=1.5) [invalido]: Parametro invalido
P Poisson(k=2, lambda=-1) [invalido]: Parametro invalido
=====Covariância, Correlação e Coeficiente Angular de Regressão
   =====
Covariância: 9.250000
Covariância com n=0 [invalido]: N invalido
```

```
Correlação: 0.968334
Correlação com n=0 [invalido]: N invalido
Coeficiente Angular de Regressão: 1.267123
Coeficiente Angular de Regressão com n=0 [invalido]: N invalido
```

7.5 Versões estática e dinâmica

O uso e a compilação de bibliotecas estáticas e dinâmicas são demonstrados no `Makefile`. A aplicação cliente é compilada duas vezes: uma com a biblioteca estática (`libestat.a`) e outra com a biblioteca dinâmica (`libestat.so`).

No executável compilado pela biblioteca estática, a biblioteca é incorporada ao executável durante a compilação da aplicação, assim permitindo que a biblioteca seja utilizada mesmo que não esteja presente no sistema do usuário.

No executável compilado pela biblioteca dinâmica, o código não é incorporado: o executável guarda apenas a referência à biblioteca, e o carregador dinâmico a procura quando o programa é executado. Se ele não a encontrar, ocorre um erro de carregamento da biblioteca. Para evitar isso, a pasta onde ela está pode ser indicada pela variável de ambiente `LD_LIBRARY_PATH` ou gravada no próprio executável na hora da ligação, com a opção `-Wl,-rpath`. Já as opções `-L` e `-l` são usadas apenas na compilação, para que o linker localize a biblioteca.

Os executáveis gerados são `bin/app` (estático) e `bin/app_dyn` (dinâmico), e a solução adotada pelo grupo é a variável `LD_LIBRARY_PATH=lib`. A parte dinâmica foi verificada em Linux, e não no macOS. Sem a variável, o `ldd` não encontra a biblioteca e a execução falha. Com a variável, o `ldd` a localiza em `lib/` (trecho relevante da saída):

```
$ ldd bin/app_dyn
        libestat.so => not found
        libm.so.6 => /lib64/libm.so.6
        libc.so.6 => /lib64/libc.so.6

$ LD_LIBRARY_PATH=lib ldd bin/app_dyn
        libestat.so => lib/libestat.so
        libm.so.6 => /lib64/libm.so.6
        libc.so.6 => /lib64/libc.so.6

$ ./bin/app_dyn
./bin/app_dyn: error while loading shared libraries: libestat.so:
   cannot open shared object file: No such file or directory
```

8 Compilação e Distribuição

8.1 Makefile

O Makefile usa variáveis para os diretórios (`SRC_DIR`, `BUILD_DIR`, `LIB_DIR`, `BIN_DIR`) e regras genéricas com `wildcard` e `%`, tal que um novo arquivo em `src/` ou `tests/` é incluído sem precisar alterar o Makefile.

Alvo	Função
`make`	gera `lib/libestat.a` e `lib/libestat.so`
`make test`	compila e executa os testes com a biblioteca estática
`make test-dyn`	compila e executa os testes com a biblioteca dinâmica
`make app`	gera `bin/app` (estático)
`make app-dyn`	gera `bin/app_dyn` (dinâmico)
`make clean`	remove os arquivos gerados
`make run`	compila e executa `bin/app` (estático)
`make run-dyn`	compila e executa `bin/app_dyn` com `LD_LIBRARY_PATH`
`make install`	instala bibliotecas e cabeçalhos no diretório definido por `PREFIX`

Tabela 20: Alvos do Makefile.

8.2 Biblioteca estática e dinâmica

- Estática: os `.c` viram objetos (`.o`) e são empacotados com `ar rcs lib/libestat.a build/*.o`. O código usado é copiado para o executável, que não depende da biblioteca na execução.
- Dinâmica: os objetos são compilados com `-fPIC` (código independente de posição) e unidos com `gcc -shared`. O executável guarda só a referência, e a biblioteca é carregada na execução.
- Ambas são ligadas com `-lm`, colocado no final do comando por causa da ordem de resolução dos símbolos.

8.3 Compilação incremental

Como os objetos dependem do seu `.c` e dos cabeçalhos, alterar um `.c` recompila apenas ele e refaz as bibliotecas.

```
gcc -Wall -Wextra -Iinclude -c src/esperanca.c -o build/esperanca.o
ar rcs lib/libestat.a build/correlacao.o build/distribuicoes.o ...
gcc -Wall -Wextra -Iinclude -fPIC -c src/esperanca.c -o build/pic/
   esperanca.o
gcc -shared build/pic/correlacao.o ... -o lib/libestat.so -lm
```

8.4 Localização da biblioteca dinâmica

Na ligação, `-L` indica o diretório da biblioteca e `-l` o seu nome (`-lestat` procura `libestat.so`). Porém, na execução o carregador não procura em `lib/`, e o programa falha com erro de biblioteca não encontrada. A solução adotada é a variável de ambiente:

```
LD_LIBRARY_PATH=lib ./bin/app_dyn
```

Alternativamente, o caminho pode ser gravado no executável com `-Wl,-rpath`.

8.5 Instalação da biblioteca

Para permitir que a biblioteca seja utilizada por programas externos ao diretório do projeto, foi implementado o alvo `install` no Makefile.

Esse alvo instala as duas versões da biblioteca, estática e dinâmica, juntamente com os cabeçalhos públicos necessários para sua utilização.

O diretório de instalação é configurado pela variável `PREFIX`, cujo valor padrão é o diretório `$HOME/.local`. Dessa maneira, a instalação pode ser realizada no diretório pessoal do usuário, sem necessidade de privilégios administrativos.

O comando utilizado para verificar a instalação foi:

```
make install PREFIX="$HOME/.local"
```

O alvo depende das bibliotecas `libestat.a` e `libestat.so`. Portanto, caso elas ainda não tenham sido compiladas, o Makefile realiza sua construção antes da instalação.

Os arquivos são organizados da seguinte maneira:

- `$HOME/.local/lib/libestat.a`: biblioteca estática;
- `$HOME/.local/lib/libestat.so`: biblioteca dinâmica;
- `$HOME/.local/include/`: diretório que recebe os cinco cabeçalhos públicos.

Para a instalação, foram utilizados os comandos `install -d`, responsável pela criação dos diretórios necessários, e `install -m`, responsável pela cópia dos arquivos com as permissões definidas.

As bibliotecas e os cabeçalhos foram instalados com sucesso no diretório pessoal do usuário.

8.5.1 Verificação da instalação

Após a instalação, foi criado um pequeno programa C externo ao projeto para verificar se a biblioteca poderia ser utilizada independentemente da estrutura original dos arquivos-fonte.

O programa incluiu os cabeçalhos `correlacao.h` e `erros.h` e realizou uma chamada à função `correlacao` com as amostras $X = \{1, 2, 3\}$ e $Y = \{3, 5, 7\}$.

O resultado esperado era uma correlação igual a 1, acompanhada do código `ESTAT_OK`.

Primeiramente, o programa foi compilado utilizando a biblioteca estática instalada:

```
gcc -Wall -Wextra -std=c11 \
    -I"$HOME/.local/include" \
    /tmp/teste_libestat_instalada.c \
    "$HOME/.local/lib/libestat.a" \
    -lm -o /tmp/teste_libestat_instalada

/tmp/teste_libestat_instalada
```

A execução produziu a seguinte saída:

```
Correlacao: 1.000000
Codigo de erro: 0
```

Em seguida, o mesmo programa foi compilado com a biblioteca dinâmica instalada:

```
gcc -Wall -Wextra -std=c11 \
    -I"$HOME/.local/include" \
    /tmp/teste_libestat_instalada.c \
    -L"$HOME/.local/lib" -lestat \
```

```
    -lm -o /tmp/teste_libestat_dinamica

LD_LIBRARY_PATH="$HOME/.local/lib" \
    /tmp/teste_libestat_dinamica
```

A execução dinâmica também apresentou o resultado esperado:

```
Correlacao: 1.000000
Codigo de erro: 0
```

Por fim, o comando `ldd` foi utilizado para confirmar que o executável dinâmico estava carregando a biblioteca compartilhada instalada:

```
LD_LIBRARY_PATH="$HOME/.local/lib" \
    ldd /tmp/teste_libestat_dinamica \
    | grep libestat
```

O resultado confirmou que `libestat.so` foi localizada no diretório `$HOME/.local/lib`.

Esses testes demonstraram que o alvo `install` permite disponibilizar a biblioteca para programas externos, tanto por ligação estática quanto por ligação dinâmica.

9 Documentação com Doxygen

9.1 Convenções de comentário

Toda função pública é documentada no cabeçalho (`.h`), e não no `.c`, porque é o cabeçalho que o Doxygen processa e que o usuário da biblioteca lê. Os comentários começam com `/**` (dois asteriscos), que é o que os distingue de um comentário comum. Cada cabeçalho tem um bloco `@file` com `@brief` descrevendo o módulo, e cada função usa:

- `@brief`: uma linha dizendo o que a função calcula;
- `@param`: um por parâmetro, com o mesmo nome da assinatura (incluindo `erro`);
- `@return`: o valor devolvido, inclusive o que ocorre em caso de erro;
- `@note`: o código de erro associado a cada condição inválida.

Exemplo, retirado de `include/distribuicoes.h`:

```
/**
 * @brief Calcula P(X = k) para X ~ Binomial(n, p).
 * @param k Quantidade de sucessos.
 * @param n Quantidade de ensaios.
 * @param p Probabilidade de sucesso em cada ensaio (0 <= p <= 1).
 * @param erro Ponteiro onde o codigo de erro sera escrito (
    ESTAT_OK em sucesso).
 * @return P(X = k), ou 0.0 em caso de erro. Retorna 0.0 valido se
    k > n.
 * @note *erro = ESTAT_ERRO_N_INVALIDO se n <= 0 ou k < 0.
 * @note *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se p fora de [0,1].
 */
double probabilidade_binomial(int k, int n, double p, int *erro);
```

Os códigos de erro citados nos `@note` são os da Seção 3.3.

9.2 Geração

A documentação é gerada a partir dos cabeçalhos de `include/`, com o arquivo de configuração `Doxyfile` da raiz do projeto, criado com `doxygen -g` e ajustado pelo grupo (versão 1.18.0 do Doxygen):

```
doxygen Doxyfile
xdg-open docs/html/index.html
```

As opções alteradas em relação ao arquivo padrão são:

- `PROJECT_NAME = "lib_estat"` e `INPUT = include README.md`: apenas os cabeçalhos públicos e o README são processados;
- `OUTPUT_DIRECTORY = docs`: a saída HTML fica em `docs/html`;
- `OUTPUT_LANGUAGE = Brazilian`: páginas em português;
- `OPTIMIZE_OUTPUT_FOR_C = YES`: apresentação própria para C;
- `GENERATE_TREEVIEW = YES`: menu lateral em árvore;
- `GENERATE_LATEX = NO`: apenas a saída em HTML;
- `SHOW_INCLUDE_FILES = NO`: a lista de `#include` não aparece no topo da página de cada arquivo;
- `EXTRACT_ALL = YES`: o Doxygen documenta todos os elementos, mesmo os que ainda não têm comentário.
- `USE_MDFILE_AS_MAINPAGE = README.md`: o README é usado como página principal da documentação;

O `Doxyfile` é versionado, mas a pasta `docs/html` é saída gerada e está no `.gitignore`; qualquer integrante a reproduz com o comando acima.

9.3 Verificação

A geração sobre o diretório `include/` processou os cinco cabeçalhos do projeto (`correlacao.h`, `distribuicoes.h`, `erros.h`, `esperanca.h` e `variancia_modelos.h`) e terminou sem nenhum aviso, com `WARN_IF_DOC_ERROR` e `WARN_NO_PARAMDOC` ativos. Isso garante que os `@param` dos comentários existentes batem com os parâmetros reais das funções e que nenhum parâmetro documentado ficou sem descrição.

Como o `EXTRACT_ALL` está ativo, o Doxygen não avisa sobre elementos sem comentário; por isso, a completude dos comentários (`@brief`, `@param`, `@return`) foi conferida também na leitura das páginas HTML geradas.

Durante essa verificação, foi detectada e corrigida uma declaração duplicada de `variancia_uniforme` no cabeçalho do módulo de variância. Durante a geração também foi necessário corrigir duas linhas do `Doxyfile` que tinham ficado com o valor inválido por erro em um comando `sed` (`WARN_IF_UNDOCUMENTED` e `WARN_NO_PARAMDOC`); o Doxygen apontou o problema ao avisar que o argumento não era um valor booleano válido.



Figura 1: Página inicial da documentação, gerada a partir do `README.md`: introdução, tabela de módulos e menu lateral com as seções.

10 Controle de Versão

10.1 Fluxo de branches

O desenvolvimento do projeto foi organizado utilizando o sistema de controle de versão Git, com o repositório remoto hospedado no GitHub.

O grupo adotou uma estratégia de desenvolvimento baseada em branches, permitindo que cada integrante implementasse suas funcionalidades de maneira independente, sem modificar diretamente a versão principal do projeto.

A branch `main` foi reservada para a versão estável destinada à entrega final, enquanto a branch `develop` foi utilizada para integrar e testar as contribuições dos integrantes.

Para o desenvolvimento do módulo de correlação, foi criada a branch `feature/correlacao-implemen` Nela foram implementadas as funções de covariância, correlação de Pearson e regressão linear simples, além dos respectivos testes automatizados.

Após a implementação e a execução dos testes, as alterações foram registradas em um commit e enviadas ao repositório remoto. Em seguida, foi criado um Pull Request para integrar a funcionalidade à branch `develop`.

Posteriormente, foi criada a branch `feature/make-install`, destinada à implementação do alvo `install` no Makefile.

Essa funcionalidade permitiu instalar as bibliotecas estática e dinâmica, juntamente com os cabeçalhos públicos, em um diretório configurável por meio da variável `PREFIX`.

Após a validação da instalação e dos testes com programas externos, as alterações também foram integradas à branch `develop` por meio de um Pull Request.

A documentação das contribuições foi realizada separadamente, na branch `docs/relatorio-correla`

Esse fluxo de trabalho permitiu separar o desenvolvimento individual da integração do projeto, facilitando a revisão das alterações e a manutenção de um histórico organizado.

A integração final da branch `develop` à `main`, juntamente com a criação da tag `v1.0`, foi planejada para a etapa de finalização e entrega do projeto. `v1.0`, foi planejada para a etapa de finalização e entrega do projeto.

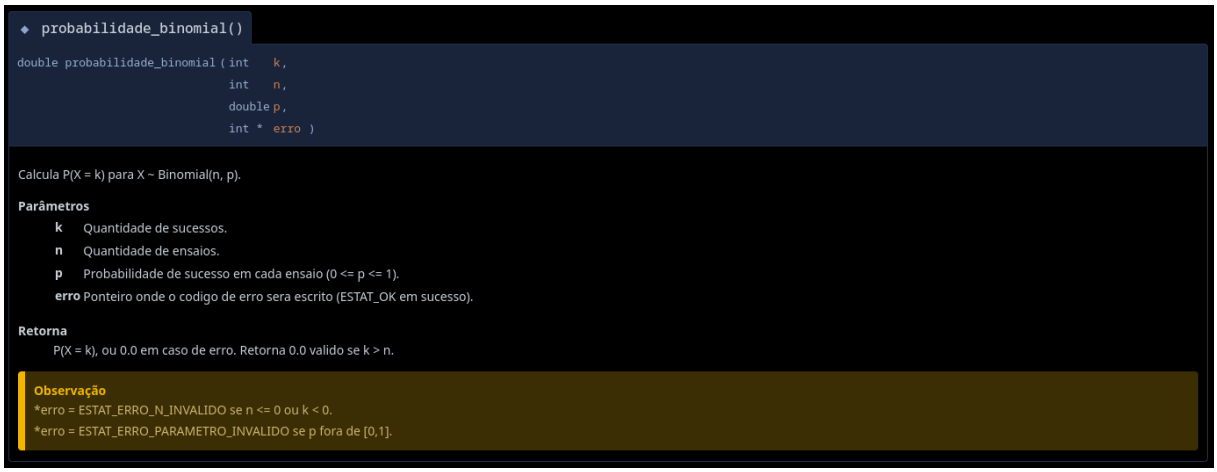


Figura 2: Descrição de `probabilidade_binomial` na página de `distribuicoes.h` gerada pelo Doxygen: parâmetros, retorno e notas.

10.2 Histórico

O histórico do repositório registra o desenvolvimento incremental da biblioteca e a integração das contribuições dos integrantes do grupo.

Para visualizar os commits e as relações entre as branches, foi utilizado o comando:

```
git log --oneline --graph --decorate --all
```

A Tabela a seguir apresenta alguns dos commits representativos do desenvolvimento.

Commit	Descrição
`79d42e4`	Implementação do módulo de correlação, incluindo as três funções públicas e os testes automatizados.
`1ba33a9`	Correção da convenção e do tratamento de erros da distribuição geométrica no módulo de esperança.
`ca2e2af`	Adição dos alvos `run` e `run-dyn` ao Makefile.
`46d2c19`	Ampliação dos testes de distribuições e padronização da função de aproximação numérica.
`0daa315`	Preenchimento de partes do relatório em LaTeX.
`4f4ec94`	Adição das saídas dos testes e da aplicação cliente ao relatório.
`7041eb9`	Complementação das seções de distribuições, testes automatizados e documentação Doxygen.
`9033ddc`	Implementação do alvo `install` com a variável `PREFIX` configurável.
`212816d`	Integração da funcionalidade de instalação à branch `develop`, por meio do Pull Request número 6.

Tabela 21: Exemplos de commits registrados no histórico do projeto.

O desenvolvimento em branches individuais permitiu que diferentes módulos fossem implementados em paralelo.

As integrações foram realizadas na branch `develop`, reunindo implementações, testes, documentação e melhorias no sistema de compilação.

A utilização de commits com mensagens descritivas facilitou a identificação das alterações realizadas e o acompanhamento da evolução do projeto.

10.3 Conflitos e como foram resolvidos

As funcionalidades do módulo de correlação e do alvo `install` foram desenvolvidas em branches próprias e integradas à branch `develop` por meio de Pull Requests.

Nessas duas contribuições, não foi necessário realizar uma resolução manual de conflitos durante a integração.

O desenvolvimento em branches separadas permitiu que as alterações fossem revisadas antes de serem incorporadas à versão de integração.

Essa organização também reduziu o risco de sobrescrever modificações realizadas por outros integrantes do grupo.

O histórico registra ainda atualizações entre branches de desenvolvimento, como integrações da `develop` à branch `readme` e o posterior merge desta à `develop`.

10.4 Versão `v1.0`

11 Dificuldades e Soluções

Dificuldade	Solução
Convenção da geométrica diferente entre módulos	A esperança usava $1/p$ (número de tentativas), divergindo dos demais módulos (número de falhas), e $p = 0$ gerava códigos de erro diferentes. O grupo adotou k como número de falhas ($E[X] = (1-p)/p$) e `ESTAT_ERRO_DIVISAO_ZERO` para $p = 0$ em ambos os módulos. A correção foi aplicada à esperança (commit `1ba33a9`).
Declaração duplicada no cabeçalho de variância	Detectada pelo Doxygen e removida
GDB indisponível no macOS com chip Apple	Sessão feita em máquina Linux; ensaio prévio com `lldb`
Nome da biblioteca gerada (`liblib_estat`), com o prefixo `lib` duplicado	Corrigido no Makefile (`LIB_NAME = estat`), que passou a gerar `libestat.a` e `libestat.so`
Biblioteca dinâmica não encontrada na execução	Uso de `LD_LIBRARY_PATH=lib` (alternativa: `-Wl,-rpath`)

Tabela 22: Principais problemas encontrados.

12 Conclusão

O projeto entregou uma biblioteca em C com 15 funções públicas distribuídas em quatro módulos (esperança, variância, distribuições de probabilidade e correlação), com tratamento de erros padronizado pelo parâmetro `int *erro`. Também foram entregues as versões estática (`libestat.a`) e dinâmica (`libestat.so`), geradas por um Makefile, os

testes automatizados de todos os módulos, uma aplicação cliente que demonstra o uso das funções e a documentação gerada com Doxygen.

Durante o trabalho, o grupo aprendeu a padronizar convenções entre módulos (como a da distribuição geométrica e o código de erro para $p = 0$), a comparar números em ponto flutuante com tolerância, a diferença entre ligação estática e dinâmica e a usar branches para integrar o trabalho de várias pessoas. Como melhorias futuras, a biblioteca poderia incluir novos modelos, como a Normal e a Exponencial, e a parte dinâmica poderia ser validada também no macOS.

A Código-fonte

A.1 include/correlacao.h

```
1
2  #ifndef CORRELACAO_H
3  #define CORRELACAO_H
4
5  #include "erros.h"
6
7  /**
8   * @file correlacao.h
9   * @brief Modulo de correlacao e regressao linear.
10  *
11  * Funcoes para analise estatistica de duas amostras observadas.
12  */
13
14 /**
15  * @brief Calcula a covariancia amostral entre duas variaveis.
16  *
17  * @param amostra_x Array com os valores da primeira amostra.
18  * @param amostra_y Array com os valores da segunda amostra.
19  * @param n Numero de observacoes em cada amostra (n > 1).
20  * @param erro Ponteiro para armazenar o codigo de erro.
21  *
22  * @return Valor da covariancia amostral.
23  *         Retorna 0.0 em caso de erro.
24  *
25  * @note ESTAT_OK: calculo realizado com sucesso.
26  * @note ESTAT_ERRO_N_INVALIDO: n menor ou igual a 1.
27  * @note ESTAT_ERRO_PARAMETRO_INVALIDO: algum array e NULL.
28  * @note Se erro for NULL, nenhum codigo de erro sera armazenado.
29  */
30 double covariancia(double amostra_x[], double amostra_y[], int n,
       int *erro);
31
32 /**
33  * @brief Calcula o coeficiente de correlacao de Pearson.
34  *
35  * Mede a intensidade e a direcao da relacao linear
36  * entre duas variaveis observadas.
```

```
 *
 * @param amostra_x Array com os valores da primeira amostra.
 * @param amostra_y Array com os valores da segunda amostra.
 * @param n Numero de observacoes em cada amostra (n > 1).
 * @param erro Ponteiro para armazenar o codigo de erro.
 *
 * @return Coeficiente de Pearson no intervalo [-1.0, 1.0].
 *         Retorna 0.0 em caso de erro.
 *
 * @note ESTAT_OK: calculo realizado com sucesso.
 * @note ESTAT_ERRO_N_INVALIDO: n menor ou igual a 1.
 * @note ESTAT_ERRO_PARAMETRO_INVALIDO: algum array e NULL.
 * @note ESTAT_ERRO_DIVISAO_ZERO: variancia de X ou Y igual a zero.
 * @note Se erro for NULL, nenhum codigo de erro sera armazenado.
 */
double correlacao(double amostra_x[], double amostra_y[], int n,
    int *erro);

/**
 * @brief Calcula o coeficiente angular da regressao linear simples
     .
 *
 * Calcula a inclinacao da reta de regressao de Y em funcao de X
 * pelo metodo dos minimos quadrados.
 *
 * @param amostra_x Array com os valores da variavel independente (
     X).
 * @param amostra_y Array com os valores da variavel dependente (Y)
     .
 * @param n Numero de observacoes em cada amostra (n > 1).
 * @param erro Ponteiro para armazenar o codigo de erro.
 *
 * @return Coeficiente angular da reta de regressao.
 *         Retorna 0.0 em caso de erro.
 *
 * @note ESTAT_OK: calculo realizado com sucesso.
 * @note ESTAT_ERRO_N_INVALIDO: n menor ou igual a 1.
 * @note ESTAT_ERRO_PARAMETRO_INVALIDO: algum array e NULL.
 * @note ESTAT_ERRO_DIVISAO_ZERO: variancia de X igual a zero.
 * @note Se erro for NULL, nenhum codigo de erro sera armazenado.
 */
double coeficiente_linear_regressao(double amostra_x[], double
    amostra_y[], int n, int *erro);

#endif
```

A.2 src/correlacao.c

```
#include "correlacao.h"
#include "erros.h"
```

```c
#include <stddef.h>
#include <math.h>

double covariancia(double amostra_x[], double amostra_y[], int n,
    int *erro) {
    if (erro != NULL)
        *erro = ESTAT_OK;

    if (n <= 1) {
        if (erro != NULL)
            *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }

    if (amostra_x == NULL || amostra_y == NULL) {
        if (erro != NULL)
            *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }

    double soma_x = 0.0;
    double soma_y = 0.0;

    for (int i = 0; i < n; i++) {
        soma_x += amostra_x[i];
        soma_y += amostra_y[i];
    }

    double media_x = soma_x / n;
    double media_y = soma_y / n;

    double soma_produtos = 0.0;

    for (int i = 0; i < n; i++) {
        soma_produtos += (amostra_x[i] - media_x) *
                         (amostra_y[i] - media_y);
    }

    return soma_produtos / (n - 1);
}

double correlacao(double amostra_x[], double amostra_y[], int n,
    int *erro) {
    if (erro != NULL)
        *erro = ESTAT_OK;

    if (n <= 1) {
        if (erro != NULL)
            *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }
```

```
52
53      if (amostra_x == NULL || amostra_y == NULL) {
54          if (erro != NULL)
55              *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
56          return 0.0;
57      }
58
59      double soma_x = 0.0;
60      double soma_y = 0.0;
61
62      for (int i = 0; i < n; i++) {
63          soma_x += amostra_x[i];
64          soma_y += amostra_y[i];
65      }
66
67      double media_x = soma_x / n;
68      double media_y = soma_y / n;
69
70      double soma_produtos = 0.0;
71      double soma_quadrados_x = 0.0;
72      double soma_quadrados_y = 0.0;
73
74      for (int i = 0; i < n; i++) {
75          double desvio_x = amostra_x[i] - media_x;
76          double desvio_y = amostra_y[i] - media_y;
77
78          soma_produtos += desvio_x * desvio_y;
79          soma_quadrados_x += desvio_x * desvio_x;
80          soma_quadrados_y += desvio_y * desvio_y;
81      }
82
83      if (soma_quadrados_x == 0.0 || soma_quadrados_y == 0.0) {
84          if (erro != NULL)
85              *erro = ESTAT_ERRO_DIVISAO_ZERO;
86          return 0.0;
87      }
88
89      double denominador = sqrt(soma_quadrados_x) *
90                           sqrt(soma_quadrados_y);
91
92      return soma_produtos / denominador;
93  }
94
95
96  double coeficiente_linear_regressao(double amostra_x[], double
        amostra_y[], int n, int *erro) {
97      if (erro != NULL)
98          *erro = ESTAT_OK;
99
100     if (n <= 1) {
101         if (erro != NULL)
```

```
            *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }

    if (amostra_x == NULL || amostra_y == NULL) {
        if (erro != NULL)
            *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }

    double soma_x = 0.0;
    double soma_y = 0.0;

    for (int i = 0; i < n; i++) {
        soma_x += amostra_x[i];
        soma_y += amostra_y[i];
    }

    double media_x = soma_x / n;
    double media_y = soma_y / n;

    double soma_produtos = 0.0;
    double soma_quadrados_x = 0.0;

    for (int i = 0; i < n; i++) {
        double desvio_x = amostra_x[i] - media_x;
        double desvio_y = amostra_y[i] - media_y;

        soma_produtos += desvio_x * desvio_y;
        soma_quadrados_x += desvio_x * desvio_x;
    }

    if (soma_quadrados_x == 0.0) {
        if (erro != NULL)
            *erro = ESTAT_ERRO_DIVISAO_ZERO;
        return 0.0;
    }

    return soma_produtos / soma_quadrados_x;
}
```

A.3 tests/test_correlacao.c

```

#include <assert.h>
#include <math.h>
#include <stddef.h>
#include <stdio.h>

#include "correlacao.h"
```

```c
#include "erros.h"

// compara resultados com uma pequena tolerancia numerica
static void verificar_resultado(double obtido, double esperado) {
    assert(fabs(obtido - esperado) < 1e-9);
}

// testes da funcao covariancia.
static void testar_covariancia(void) {
    double x[] = {1, 2, 3};
    double y_positivo[] = {3, 5, 7};
    double y_negativo[] = {7, 5, 3};

    int erro = -1;
    double resultado;

    // covariancia positiva
    resultado = covariancia(x, y_positivo, 3, &erro);
    verificar_resultado(resultado, 2.0);
    assert(erro == ESTAT_OK);

    // covariancia negativa
    resultado = covariancia(x, y_negativo, 3, &erro);
    verificar_resultado(resultado, -2.0);
    assert(erro == ESTAT_OK);

    // tamanho de amostra invalido
    resultado = covariancia(x, y_positivo, 1, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_N_INVALIDO);

    // ponteiro de amostra invalido
    resultado = covariancia(NULL, y_positivo, 3, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);

    printf("testes de covariancia: OK\n");
}

// teste da funcao correlacao
static void testar_correlacao(void) {
    double x[] = {1, 2, 3};
    double y_positivo[] = {3, 5, 7};
    double y_negativo[] = {7, 5, 3};
    double constante[] = {4, 4, 4};

    int erro = -1;
    double resultado;

    // correlacao positiva perfeita
    resultado = correlacao(x, y_positivo, 3, &erro);
```

```
    verificar_resultado(resultado, 1.0);
    assert(erro == ESTAT_OK);

    // correlacao negativa perfeita
    resultado = correlacao(x, y_negativo, 3, &erro);
    verificar_resultado(resultado, -1.0);
    assert(erro == ESTAT_OK);

    // variancia zero em X
    resultado = correlacao(constante, y_positivo, 3, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_DIVISAO_ZERO);

    // variancia zero em Y
    resultado = correlacao(x, constante, 3, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_DIVISAO_ZERO);

    // tamanho invalido
    resultado = correlacao(x, y_positivo, 0, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_N_INVALIDO);

    // amostra invalida
    resultado = correlacao(x, NULL, 3, &erro);
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);

    printf("testes de correlacao: OK\n");
}

// teste da funcao de regressao linear
static void testar_regressao(void) {
    double x[] = {1, 2, 3};
    double y_positivo[] = {3, 5, 7};
    double y_negativo[] = {7, 5, 3};
    double constante[] = {4, 4, 4};

    int erro = -1;
    double resultado;

    // coeficiente angular positivo
    resultado = coeficiente_linear_regressao(
        x, y_positivo, 3, &erro
    );
    verificar_resultado(resultado, 2.0);
    assert(erro == ESTAT_OK);

    // coeficiente angular negativo
    resultado = coeficiente_linear_regressao(
        x, y_negativo, 3, &erro
```

```
    );
    verificar_resultado(resultado, -2.0);
    assert(erro == ESTAT_OK);

    // coeficiente angular zero e resultado valido
    resultado = coeficiente_linear_regressao(
        x, constante, 3, &erro
    );
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_OK);

    // variancia zero em X
    resultado = coeficiente_linear_regressao(
        constante, y_positivo, 3, &erro
    );
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_DIVISAO_ZERO);

    // tamanho invalido
    resultado = coeficiente_linear_regressao(
        x, y_positivo, 1, &erro
    );
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_N_INVALIDO);

    // ponteiro invalido
    resultado = coeficiente_linear_regressao(
        NULL, y_positivo, 3, &erro
    );
    verificar_resultado(resultado, 0.0);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);

    printf("testes de regressao: OK\n");
}

int main(void) {
    testar_covariancia();
    testar_correlacao();
    testar_regressao();

    printf("todos os testes do modulo passaram!\n");
    return 0;
}
```

A.4 include/variancia_modelos.h

```
#ifndef VARIANCIA_MODELOS_H
#define VARIANCIA_MODELOS_H


```

```c
#include "erros.h"

/**
 * @file variancia_modelos.h
 * @brief Módulo de Cálculo de Variância.
 */

/**
 * @brief Função que calcula a variância de uma distribuição binomial
 *
 *
 * @param n Parâmentro n que diz a quantidade de ensaios independetes da distribuição binomial
 * @param p Parâmetro p que diz a probabilidade de sucesso em cada ensaio independente da binomial
 * @param erro Ponteiro para onde o código de erro será escrito
 *             (ESTAT_OK em caso de sucesso).
 *
 * @return A função retorna a variância da binomial ou 0.0 em caso
 *         de erro (verificar *erro para o motivo).
 *
 * @note Define *erro = ESTAT_ERRO_N_INVALIDO se n <= 0.
 * @note Define *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se p fora de [0,1].
 */
double variancia_binomial(int n, double p, int *erro);

/**
 * @brief Função que calcula a variância de uma distribuição geometrica
 *
 *
 * @param p Parâmetro p que diz a probabilidade de ocorrência do evento desejado
 * @param erro Ponteiro para onde o código de erro será escrito
 *             (ESTAT_OK em caso de sucesso).
 *
 * @return A função retorna a variância da geométrica ou 0.0 em caso
 *         de erro (verificar *erro para o motivo).
 *
 * @note Define *erro = ESTAT_ERRO_DIVISAO_ZERO se p=0
 * @note Define *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se p fora de [0,1].
 */
double variancia_geometrica(double p, int *erro);

/**
 * @brief Função que calcula a variância de uma distribuiçao de variáveis contínuas em um intervalo
```

```
 *
 *
 * @param a Menor número do intervalo
 * @param b Maior número do intervalo
 * @param erro Ponteiro para onde o código de erro será escrito
 *             (ESTAT_OK em caso de sucesso).
 *
 * @return A função retorna a variância da uniforme ou 0.0 em caso
 *         de erro (verificar *erro para o motivo).
 *
 * @note A distribuição é para variáveis contínuas em um intervalo
    [a,b].
 * @note Define *erro=ESTAT_ERRO_PARAMETRO_INVALIDO se a>=b.
 */
double variancia_uniforme(double a, double b, int *erro);


/**
 * @brief Função que calcula a variância de uma distribuição
    Poisson
 *
 *
 * @param lambda Parametro lambda que diz a taxa de ocorrência do
    evento que se deseja modelar
 * @param erro Ponteiro para onde o código de erro será escrito
 *             (ESTAT_OK em caso de sucesso).
 *
 * @return A função retorna a variância da Poisson ou 0.0 em caso
 *         de erro (verificar *erro para o motivo).
 *
 * @note Define *erro=ESTAT_ERRO_PARAMETRO_INVALIDO se lambda<=0
 */
double variancia_poisson(double lambda, int *erro);




#endif
```

A.5 src/variancia_modelos.c

```
#include "variancia_modelos.h"

double variancia_binomial(int n, double p, int *erro){
    if (n <= 0) {
        *erro = ESTAT_ERRO_N_INVALIDO;
        return 0.0;
    }
    if (p < 0 || p > 1) {
        *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
```

```
        return 0.0;
    }

    *erro = ESTAT_OK;
    return n*p*(1-p);

}
double variancia_geometrica(double p, int *erro){
    if (p<0 || p>1){
        *erro=ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }
    if(p==0){
        *erro=ESTAT_ERRO_DIVISAO_ZERO;
        return 0.0;
    }
    *erro=ESTAT_OK;
    return (1-p)/(p*p);
}
double variancia_uniforme(double a, double b, int *erro){
    if (a>=b) {
        *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }

    *erro = ESTAT_OK;
    return ((b-a)*(b-a))/12;
}

double variancia_poisson(double lambda, int *erro){
    if (lambda <= 0) {
        *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
        return 0.0;
    }
    *erro = ESTAT_OK;
    return lambda;
}
```

A.6 tests/test_variancia_modelos.c

```
#include "variancia_modelos.h"
#include <assert.h>
#include <math.h>
#include <stdio.h>

//função que permite a checagem de aproximação com double
int aproximadamente(double x, double y){
    return fabs(y-x)<0.000001;
}
//testes de variancia binomial
```

```c
void teste_variancia_binomial_erro_n_invalido(){
    int erro;
    variancia_binomial(-2, 0.5, &erro);
    assert(erro == ESTAT_ERRO_N_INVALIDO);
    printf("passou no teste_variancia_binomial_erro_n_invalido\n");
}

void teste_variancia_binomial_erro_p_invalido(){
    int erro;
    variancia_binomial(10, -1, &erro);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
    printf("passou no teste_variancia_binomial_erro_p_invalido\n");
}
void teste_variancia_binomial_normal(){
    int erro;
    double res = variancia_binomial(10, 0.5, &erro);
    assert(erro == ESTAT_OK);
    assert(aproximadamente(res, 2.5));
    printf("passou no teste_variancia_binomial_normal\n");}

void teste_variancia_binomial_p_igual_1(){
    int erro;
    double res = variancia_binomial(10, 1, &erro);
    assert(erro == ESTAT_OK);
    assert(aproximadamente(res, 0));
    printf("passou no teste_variancia_binomial_p_igual_1\n");
}
void teste_variancia_binomial_n_igual_0(){
    int erro;
    variancia_binomial(0, 0.5, &erro);
    assert(erro == ESTAT_ERRO_N_INVALIDO);
    printf("passou no teste_variancia_binomial_n_igual_0\n");
}
void teste_variancia_binomial_p_igual_0(){
    int erro;
    double res = variancia_binomial(10, 0, &erro);
    assert(erro == ESTAT_OK);
    assert(aproximadamente(res, 0));
    printf("passou no teste_variancia_binomial_p_igual_0\n");
}

//teste de variancia geométrica
void teste_variancia_geo_erro_div_por_zero(){
    int erro;
    variancia_geometrica(0,&erro);
    assert(erro==ESTAT_ERRO_DIVISAO_ZERO);
    printf("passou no teste_variancia_geo_erro_div_por_zero\n");

}

void teste_variancia_geo_erro_p_invalido(){
```

```
62      int erro;
63      variancia_geometrica(2,&erro);
64      assert(erro==ESTAT_ERRO_PARAMETRO_INVALIDO);
65      printf("passou no teste_variancia_geo_erro_p_invalido\n");
66  }
67  void teste_variancia_geo_normal(){
68      int erro;
69      double res=variancia_geometrica(0.5,&erro);
70      assert(erro==ESTAT_OK);
71      assert(aproximadamente(res,2));
72      printf("passou no teste_variancia_geo_normal\n");
73  }
74  void teste_variancia_geo_p_igual_1(){
75      int erro;
76      double res=variancia_geometrica(1,&erro);
77      assert(erro==ESTAT_OK);
78      assert(aproximadamente(res,0));
79      printf("passou no teste_variancia_geo_p_igual_1\n");
80  }
81
82  //testes para variancia de poisson
83  void teste_variancia_poisson_lambda_invalido(){
84      int erro;
85      variancia_poisson(0,&erro);
86      assert(erro==ESTAT_ERRO_PARAMETRO_INVALIDO);
87      printf("passou no teste_variancia_poisson_lambda_invalido\n");
88  }
89  void teste_variancia_poisson_normal(){
90      int erro;
91      double res=variancia_poisson(10,&erro);
92      assert(erro==ESTAT_OK);
93      assert(aproximadamente(res,10));
94      printf("passou no teste_variancia_poisson_normal\n");
95  }
96  //testes para variancia uniforme
97  void teste_variancia_uniforme_erro_a_maior_ou_igual_b(){
98      int erro;
99      variancia_uniforme(5,5,&erro);
100     assert(erro==ESTAT_ERRO_PARAMETRO_INVALIDO);
101     printf("passou no
            teste_variancia_uniforme_erro_a_maior_ou_igual_b\n");
102 }
103 void teste_variancia_uniforme_normal(){
104     int erro;
105     double res=variancia_uniforme(0,10,&erro);
106     assert(erro==ESTAT_OK);
107     assert(aproximadamente(res,8.333333));
108     printf("passou no teste_variancia_uniforme_normal\n");
109 }
110 void teste_variancia_uniforme_a_negativo(){
111     int erro;
```

```
112     double res=variancia_uniforme(-5,5,&erro);
113     assert(erro==ESTAT_OK);
114     assert(aproximadamente(res,8.333333));
115     printf("passou no teste_variancia_uniforme_a_negativo\n");
116 }
117 
118 int main(){
119     //testes de variancia binomiais
120     teste_variancia_binomial_erro_n_invalido();
121     teste_variancia_binomial_erro_p_invalido();
122     teste_variancia_binomial_normal();
123     teste_variancia_binomial_p_igual_1();
124     teste_variancia_binomial_n_igual_0();
125     teste_variancia_binomial_p_igual_0();
126     //testes de variancia geometrica
127     teste_variancia_geo_erro_div_por_zero();
128     teste_variancia_geo_erro_p_invalido();
129     teste_variancia_geo_normal();
130     teste_variancia_geo_p_igual_1();
131     //testes de variancia poisson
132     teste_variancia_poisson_lambda_invalido();
133     teste_variancia_poisson_normal();
134     //testes de variancia uniforme
135     teste_variancia_uniforme_erro_a_maior_ou_igual_b();
136     teste_variancia_uniforme_normal();
137     teste_variancia_uniforme_a_negativo();
138     return 0;
139 }
```

A.7 include/distribuicoes.h

```
1  #ifndef DISTRIBUICOES_H
2  #define DISTRIBUICOES_H
3  
4  #include "erros.h"
5  
6  /**
7   * @file distribuicoes.h
8   * @brief Módulo de Distribuição de Probabilidade.
9   */
10 
11 /**
12  * @brief Calcula P(X = k) para X ~ Binomial(n, p).
13  * @param k Quantidade de sucessos.
14  * @param n Quantidade de ensaios.
15  * @param p Probabilidade de sucesso em cada ensaio (0 <= p <= 1).
16  * @param erro Ponteiro onde o codigo de erro sera escrito (
        ESTAT_OK em sucesso).
17  * @return P(X = k), ou 0.0 em caso de erro. Retorna 0.0 valido se
        k > n.
```

```
 * @note *erro = ESTAT_ERRO_N_INVALIDO se n <= 0 ou k < 0.
 * @note *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se p fora de [0,1].
 */
double probabilidade_binomial(int k, int n, double p, int *erro);

/**
 * @brief Calcula P(X = k) para X ~ Geometrica(p), com k fracassos antes do primeiro sucesso.
 * @param k Quantidade de fracassos antes do sucesso.
 * @param p Probabilidade de sucesso (0 < p <= 1).
 * @param erro Ponteiro onde o codigo de erro sera escrito.
 * @return P(X = k), ou 0.0 em caso de erro.
 * @note *erro = ESTAT_ERRO_N_INVALIDO se k < 0.
 * @note *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se p <= 0 ou p > 1.
 */
double probabilidade_geometrica(int k, double p, int *erro);

/**
 * @brief Calcula P(X = k) para X ~ Poisson(lambda).
 * @param k Quantidade de ocorrencias no intervalo.
 * @param lambda Taxa de ocorrencia (lambda > 0).
 * @param erro Ponteiro onde o codigo de erro sera escrito.
 * @return P(X = k), ou 0.0 em caso de erro.
 * @note *erro = ESTAT_ERRO_N_INVALIDO se k < 0.
 * @note *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se lambda <= 0.
 */
double probabilidade_poisson(int k, double lambda, int *erro);

/**
 * @brief Calcula a densidade f(x) de X ~ Uniforme(a, b).
 * @param x Ponto onde a densidade e avaliada.
 * @param a Limite inicial do intervalo.
 * @param b Limite final do intervalo.
 * @param erro Ponteiro onde o codigo de erro sera escrito.
 * @return 1/(b-a) se a <= x <= b; 0.0 fora do intervalo (retorno valido, nao e erro).
 * @note *erro = ESTAT_ERRO_PARAMETRO_INVALIDO se a >= b.
 * @note f(x) e densidade, nao probabilidade pontual (P(X = x) = 0).
 */
double densidade_uniforme(double x, double a, double b, int *erro);

#endif
```

A.8 src/distribuicoes.c

```
#include "distribuicoes.h"
#include <math.h>

// Calcula C(n, k) iterativamente a fim de evitar overflow
```

```c
static double binomio(double n, double k){
  if (k > n - k) k = n - k;
  double c = 1.0;
  for (int i = 1; i <= k; i++) {
    c = c * (n - k + i ) / i;
}
return c;
}

// Calcula P(X = k) , X ~ b(n, p)
 double probabilidade_binomial(int k, int n, double p, int *erro){
  if (n <= 0 || k < 0){
    *erro = ESTAT_ERRO_N_INVALIDO;
    return 0.0;
  }
  if (p < 0.0 || p > 1.0) {
    *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
    return 0.0;
  }

  *erro = ESTAT_OK;
  if(k > n) return 0.0;
  return binomio(n, k) * pow(p, k) * pow (1.00-p, n - k);
}

// Calcula P(X = k), X ~ G(p)
 double probabilidade_geometrica(int k, double p, int *erro){
   if( k < 0){
     *erro = ESTAT_ERRO_N_INVALIDO;
     return 0.0;
   }
   if(p <= 0.0 || p > 1.0){
    *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
    return 0.0;
  }
  *erro = ESTAT_OK;
  return p * pow((1.0-p), k);
}

// Calcula P(X = k), X ~ Po(lambda)
 double probabilidade_poisson(int k, double lambda, int *erro){
  if(k < 0){
    *erro = ESTAT_ERRO_N_INVALIDO;
    return 0.0;
  }
  if(lambda <= 0){
    *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
    return 0.0;
  }
  *erro = ESTAT_OK;
```

```c
  // exp(-lambda + k * ln(lambda) - ln(k!)) evita overflow do fatorial
  return exp(-lambda + k * log(lambda) - lgamma(k + 1.0));
}
// Calcula f(x), se X tem distribuição uniforme contínua no intervalo [a, b]
double densidade_uniforme(double x, double a, double b, int *erro) {
  if(a >= b){
    *erro = ESTAT_ERRO_PARAMETRO_INVALIDO;
    return 0.0;
  }
  *erro = ESTAT_OK;
    if(x < a || x > b) return 0.0;
    return 1.0/(b-a);
}
```

A.9 tests/test_distribuicoes.c

```c
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include "distribuicoes.h"

int aproximadamente(double x, double y) {
    return fabs(y - x) < 0.000001;
}

void test_probabilidade_binomial_p_invalida(void) {
    int erro = ESTAT_OK;
    double prob = probabilidade_binomial(1, 3, -1.0, &erro);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
    assert(prob == 0.0);
    printf("[passou] test_probabilidade_binomial_p_invalida\n");
}

void test_probabilidade_binomial_n_invalido(void) {
    int erro = ESTAT_OK;
    double prob = probabilidade_binomial(1, -1, 1.0, &erro);
    assert(erro == ESTAT_ERRO_N_INVALIDO);
    assert(prob == 0.0);
    printf("[passou] test_probabilidade_binomial_n_invalido\n");
}
// k < 0
void test_probabilidade_binomial_k_menor_que_zero(void) {
    int erro = ESTAT_OK;
    double prob = probabilidade_binomial(-1, 3, 0.5, &erro);
    assert(erro == ESTAT_ERRO_N_INVALIDO);
    assert(prob == 0.0);
```

```c
    printf("[passou] test_probabilidade_binomial_k_menor_que_zero\n
        ");
}

void test_probabilidade_binomial_k_maior_que_n(void){
  int erro = -1;
  double prob = probabilidade_binomial(4, 3, 0.5, &erro);
  assert(erro == ESTAT_OK);
  assert(prob == 0);
  printf("[passou] test_probabilidade_binomial_k_maior_que_n\n");
}

void test_probabilidade_binomial_n_zero(void){
  int erro = ESTAT_OK;
  double prob = probabilidade_binomial(0, 0, 0.5, &erro);
  assert(erro == ESTAT_ERRO_N_INVALIDO);
  assert(prob == 0.0);
  printf("[passou] test_probabilidade_binomial_n_zero\n");
}
void test_probabilidade_binomial_valida(void) {
    int erro = -1;
    /* P(X=1), n=3, p=0.5 -> 3 * 0.5^3 = 0.375 */
    double prob = probabilidade_binomial(1, 3, 0.5, &erro);
    assert(erro == ESTAT_OK);
    assert(aproximadamente(prob, 0.375));
    printf("[passou] test_probabilidade_binomial_valida\n");
}

void test_probabilidade_geometrica(void) {
    int erro = -1;
    double prob = probabilidade_geometrica(2, 0.5, &erro); /* 0.5 *
        0.25 */
    assert(erro == ESTAT_OK);
    assert(aproximadamente(prob, 0.125));
    probabilidade_geometrica(0, 0.0, &erro);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
    printf("[passou] test_probabilidade_geometrica\n");
}

void test_probabilidade_geometrica_p_invalida(void) {
    int erro = -1;
    double prob = probabilidade_geometrica(2, -0.5, &erro);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
    assert(prob == 0.0);
    printf("[passou] test_probabilidade_geometrica_p_invalida\n");
}
// k < 0
void test_probabilidade_geometrica_k_invalido(void) {
    int erro = -1;
    double prob = probabilidade_geometrica(-1, 0.5, &erro);
    assert(erro == ESTAT_ERRO_N_INVALIDO);
```

```
80      assert(prob == 0.0);
81      printf("[passou] test_probabilidade_geometrica_k_invalido\n");
82  }
83
84  void test_probabilidade_poisson(void) {
85      int erro = -1;
86      double prob = probabilidade_poisson(0, 2.0, &erro); /* e^-2 */
87      assert(erro == ESTAT_OK);
88      assert(aproximadamente(prob, exp(-2.0)));
89      probabilidade_poisson(1, -1.0, &erro);
90      assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
91      printf("[passou] test_probabilidade_poisson\n");
92  }
93
94  void test_probabilidade_poisson_lambda_invalido(void){
95    int erro = -1;
96    double prob = probabilidade_poisson(2, -1, &erro);
97    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
98    assert(prob == 0);
99    printf("[passou] test_probabilidade_poisson_lambda_invalido\n");
100 }
101 // k < 0
102 void test_probabilidade_poisson_k_invalido(void){
103   int erro = -1;
104   double prob = probabilidade_poisson(-1, 2.0, &erro);
105   assert(erro == ESTAT_ERRO_N_INVALIDO);
106   assert(prob == 0);
107   printf("[passou] test_probabilidade_poisson_k_invalido\n");
108 }
109
110 void test_densidade_uniforme(void) {
111     int erro = -1;
112     assert(aproximadamente(densidade_uniforme(1.0, 0.0, 4.0, &erro)
        , 0.25));
113     assert(erro == ESTAT_OK);
114     assert(densidade_uniforme(5.0, 0.0, 4.0, &erro) == 0.0);
115     assert(erro == ESTAT_OK);
116     densidade_uniforme(1.0, 4.0, 0.0, &erro);
117     assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
118     printf("[passou] test_densidade_uniforme\n");
119 }
120
121 void test_densidade_uniforme_a_invalido(void) { /*a >= b*/
122   int erro = -1;
123   double prob = densidade_uniforme(1, 5, 3, &erro);
124   assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
125   assert(prob == 0.0);
126   printf("[passou] test_densidade_uniforme_a_invalido\n");
127 }
128
129 void test_densidade_uniforme_a_igual_b(void){
```

```c
  int erro = ESTAT_OK;
  double prob = densidade_uniforme(2.0, 2.0, 2.0, &erro);
  assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
  assert(prob == 0.0);
  printf("[passou] test_densidade_uniforme_a_igual_b\n");
}

/* ---- testes adicionais ---- */

/* exercita o laco de binomio(): C(10,3)=120, C(10,5)=252 e o caso
   k > n-k */
void test_probabilidade_binomial_coeficiente(void) {
    int erro = -1;
    /* C(10,3) * 0.5^10 = 120/1024 */
    assert(aproximadamente(probabilidade_binomial(3, 10, 0.5, &erro
       ), 0.1171875));
    assert(erro == ESTAT_OK);
    /* C(10,5) * 0.5^10 = 252/1024 */
    assert(aproximadamente(probabilidade_binomial(5, 10, 0.5, &erro
       ), 0.24609375));
    assert(erro == ESTAT_OK);
    /* k=7 > n-k=3: usa a simetria C(10,7) = C(10,3) */
    assert(aproximadamente(probabilidade_binomial(7, 10, 0.5, &erro
       ), 0.1171875));
    assert(erro == ESTAT_OK);
    printf("[passou] test_probabilidade_binomial_coeficiente\n");
}

void test_probabilidade_binomial_p_maior_que_um(void) {
    int erro = ESTAT_OK;
    double prob = probabilidade_binomial(1, 3, 1.5, &erro);
    assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
    assert(prob == 0.0);
    printf("[passou] test_probabilidade_binomial_p_maior_que_um\n")
       ;
}

/* p = 0 e p = 1 sao validos na binomial */
void test_probabilidade_binomial_p_limites(void) {
    int erro = -1;
    assert(aproximadamente(probabilidade_binomial(0, 4, 0.0, &erro)
       , 1.0));
    assert(erro == ESTAT_OK);
    assert(aproximadamente(probabilidade_binomial(2, 4, 0.0, &erro)
       , 0.0));
    assert(erro == ESTAT_OK);
    assert(aproximadamente(probabilidade_binomial(4, 4, 1.0, &erro)
       , 1.0));
    assert(erro == ESTAT_OK);
    assert(aproximadamente(probabilidade_binomial(0, 4, 1.0, &erro)
       , 0.0));
```

```
172     assert(erro == ESTAT_OK);
173     printf("[passou] test_probabilidade_binomial_p_limites\n");
174 }
175 
176 /* n grande: 200! estouraria um double, mas o binomio iterativo
       funciona */
177 void test_probabilidade_binomial_n_grande(void) {
178     int erro = -1;
179     double prob = probabilidade_binomial(100, 200, 0.5, &erro);
180     assert(erro == ESTAT_OK);
181     assert(aproximadamente(prob, 0.056348479009256 42));
182     printf("[passou] test_probabilidade_binomial_n_grande\n");
183 }
184 
185 void test_probabilidade_geometrica_p_maior_que_um(void) {
186     int erro = -1;
187     double prob = probabilidade_geometrica(2, 1.5, &erro);
188     assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
189     assert(prob == 0.0);
190     printf("[passou] test_probabilidade_geometrica_p_maior_que_um\n
           ");
191 }
192 
193 /* p = 1: o sucesso ocorre sempre na primeira tentativa */
194 void test_probabilidade_geometrica_p_um(void) {
195     int erro = -1;
196     assert(aproximadamente(probabilidade_geometrica(0, 1.0, &erro),
           1.0));
197     assert(erro == ESTAT_OK);
198     assert(aproximadamente(probabilidade_geometrica(2, 1.0, &erro),
           0.0));
199     assert(erro == ESTAT_OK);
200     printf("[passou] test_probabilidade_geometrica_p_um\n");
201 }
202 
203 /* k > 0 exercita o lgamma: e^-3 * 3^2 / 2! */
204 void test_probabilidade_poisson_k_positivo(void) {
205     int erro = -1;
206     double prob = probabilidade_poisson(2, 3.0, &erro);
207     assert(erro == ESTAT_OK);
208     assert(aproximadamente(prob, 0.22404180765538775));
209     printf("[passou] test_probabilidade_poisson_k_positivo\n");
210 }
211 
212 void test_probabilidade_poisson_lambda_zero(void) {
213     int erro = -1;
214     double prob = probabilidade_poisson(2, 0.0, &erro);
215     assert(erro == ESTAT_ERRO_PARAMETRO_INVALIDO);
216     assert(prob == 0.0);
217     printf("[passou] test_probabilidade_poisson_lambda_zero\n");
218 }
```

```
219
220 /* k grande: com k = 200, tanto 200! quanto 200^200 estouram um
    double,
221    mas a escala logaritmica (lgamma) calcula o valor sem problemas
       */
222 void test_probabilidade_poisson_k_grande(void) {
223     int erro = -1;
224     double prob = probabilidade_poisson(200, 200.0, &erro);
225     assert(erro == ESTAT_OK);
226     assert(aproximadamente(prob, 0.028197727685920822));
227     printf("[passou] test_probabilidade_poisson_k_grande\n");
228 }
229
230 /* x = a e x = b pertencem ao intervalo [a, b] */
231 void test_densidade_uniforme_extremos(void) {
232     int erro = -1;
233     assert(aproximadamente(densidade_uniforme(0.0, 0.0, 4.0, &erro)
           , 0.25));
234     assert(erro == ESTAT_OK);
235     assert(aproximadamente(densidade_uniforme(4.0, 0.0, 4.0, &erro)
           , 0.25));
236     assert(erro == ESTAT_OK);
237     printf("[passou] test_densidade_uniforme_extremos\n");
238 }
239
240
241 int main(void) {
242     test_probabilidade_binomial_n_invalido();
243     test_probabilidade_binomial_p_invalida();
244     test_probabilidade_binomial_valida();
245     test_probabilidade_binomial_k_menor_que_zero();
246     test_probabilidade_binomial_k_maior_que_n();
247     test_probabilidade_binomial_n_zero();
248     test_probabilidade_geometrica_p_invalida();
249     test_probabilidade_geometrica_k_invalido();
250     test_probabilidade_geometrica();
251     test_probabilidade_poisson_lambda_invalido();
252     test_probabilidade_poisson_k_invalido();
253     test_probabilidade_poisson();
254     test_densidade_uniforme();
255     test_densidade_uniforme_a_invalido();
256     test_densidade_uniforme_a_igual_b();
257
258     test_probabilidade_binomial_coeficiente();
259     test_probabilidade_binomial_p_maior_que_um();
260     test_probabilidade_binomial_p_limites();
261     test_probabilidade_binomial_n_grande();
262     test_probabilidade_geometrica_p_maior_que_um();
263     test_probabilidade_geometrica_p_um();
264     test_probabilidade_poisson_k_positivo();
265     test_probabilidade_poisson_lambda_zero();
```

```
266     test_probabilidade_poisson_k_grande();
267     test_densidade_uniforme_extremos();
268 
269     return 0;
270 }
```

A.10 app/main.c

```
1  #include "variancia_modelos.h"
2  #include <stdio.h>
3  #include "erros.h"
4  #include "esperanca.h"
5  #include "distribuicoes.h"
6  #include "correlacao.h"
7  
8  const char *mensagem_erro(int codigo_erro){
9      switch(codigo_erro){
10         case ESTAT_OK:
11             return "Sem erros";
12         case ESTAT_ERRO_PARAMETRO_INVALIDO:
13             return "Parametro invalido";
14         case ESTAT_ERRO_N_INVALIDO:
15             return "N invalido";
16         case ESTAT_ERRO_DIVISAO_ZERO:
17             return "Divisao por zero";
18         case ESTAT_ERRO_TAMANHOS_DIFERENTES:
19             return "Tamanhos diferentes";
20         default:
21             return "Código de erro desconhecido";
22     }
23 }
24 void imprime_res(const char *rotulo, double valor, int codigo_erro)
       {
25     if(codigo_erro==ESTAT_OK){
26         printf("%s: %.6f\n", rotulo, valor);
27     }
28     else{
29         printf("%s: %s\n", rotulo, mensagem_erro(codigo_erro));
30     }
31 }
32 void demo_esperanca(){
33     int erro;
34     double v;
35     printf("=====Esperança dos modelos=====\n");
36     v= esperanca_binomial(10,0.5,&erro);
37     imprime_res("Esperança Binomial(10,0.5)",v,erro);
38     v=esperanca_geometrica(0.5,&erro);
39     imprime_res("Esperança Geometrica(0.5)",v,erro);
40     v=esperanca_poisson(10,&erro);
41     imprime_res("Esperança de Poisson(10)",v,erro);
```

```
42      v=esperanca_uniforme(5,10,&erro);
43      imprime_res("Esperança Uniforme(5,10)",v,erro);
44      v = esperanca_binomial(0, 0.5, &erro);
45  imprime_res("Esperança Binomial(0, 0.5) [invalido]", v, erro);
46  v = esperanca_uniforme(10, 5, &erro);
47  imprime_res("Esperança Uniforme(10, 5) [invalido]", v, erro);
48  }
49  void demo_variancia(){
50      int erro;
51      double v;
52      printf("=====Variância dos modelos=====\n");
53      v=variancia_binomial(10,0.5,&erro);
54      imprime_res("Variancia Binomial(10, 0.5)", v, erro);
55      v = variancia_geometrica(0.5, &erro);
56      imprime_res("Variancia Geometrica(0.5)", v, erro);
57      v = variancia_uniforme(0, 10, &erro);
58      imprime_res("Variancia Uniforme(0, 10)", v, erro);
59      v = variancia_poisson(4,&erro);
60      imprime_res("Variancia Poisson(4)", v, erro);
61      v = variancia_binomial(10, 1.5, &erro);
62      imprime_res("Variancia Binomial(10, 1.5) [invalido]", v, erro);
63
64  }
65  void demo_distribuicoes(){
66      double v;
67      int erro;
68      printf("=====Distribuições=====\n");
69      v = probabilidade_binomial(5, 10, 0.5, &erro);
70      imprime_res("P Binomial(k=5, n=10, p=0.5)", v, erro);
71      v = probabilidade_geometrica(3, 0.5, &erro);
72      imprime_res("P Geometrica(k=3, p=0.5)", v, erro);
73      v = probabilidade_poisson(2, 4, &erro);
74      imprime_res("P Poisson(k=2, lambda=4)", v, erro);
75      v = densidade_uniforme(5, 0, 10, &erro);
76      imprime_res("Densidade Uniforme(x=5, a=0, b=10)", v, erro);
77      v = probabilidade_binomial(5, 10, 1.5, &erro);
78      imprime_res("P Binomial(k=5, n=10, p=1.5) [invalido]", v, erro)
            ;
79      v = probabilidade_poisson(2, -1, &erro);
80      imprime_res("P Poisson(k=2, lambda=-1) [invalido]", v, erro);
81  }
82  void demo_correlacao(){
83      printf("=====Covariância, Correlação e Coeficiente Angular de
            Regressão=====\n");
84      double amostra_x[] = {2.0, 4.0, 5.0, 7.0, 9.0};
85      double amostra_y[] = {3.0, 6.0, 5.0, 9.0, 12.0};
86      int n = sizeof(amostra_x) / sizeof(amostra_x[0]);
87      int erro;
88      double cov = covariancia(amostra_x, amostra_y, n, &erro);
89      imprime_res("Covariância", cov, erro);
```

```
90      double cov_invalida = covariancia(amostra_x, amostra_y, 0, &
            erro);
91      imprime_res("Covariância com n=0 [invalido]", cov_invalida,
            erro);
92      double corr = correlacao(amostra_x, amostra_y, n, &erro);
93      imprime_res("Correlação", corr, erro);
94      double corr_invalida = correlacao(amostra_x, amostra_y, 0, &
            erro);
95      imprime_res("Correlação com n=0 [invalido]", corr_invalida,
            erro);
96      double coef_linear = coeficiente_linear_regressao(amostra_x,
            amostra_y, n, &erro);
97      imprime_res("Coeficiente Angular de Regressão", coef_linear,
            erro);
98      double coef_linear_invalido = coeficiente_linear_regressao(
            amostra_x, amostra_y, 0, &erro);
99      imprime_res("Coeficiente Angular de Regressão com n=0 [invalido
            ]", coef_linear_invalido, erro);
100 }
101 int main(){
102     demo_esperanca();
103     demo_variancia();
104     demo_distribuicoes();
105     demo_correlacao();
106
107     return 0;
108 }
```

B Saídas completas

As saídas completas do `make test` e da aplicação cliente estão nas seções *Testes Automatizados* e *Aplicação Cliente*.

C README

O repositório contém um `README.md` na raiz, com a introdução, as instruções de compilação, execução e testes, a convenção de erros e um exemplo de uso. Ele também é usado como página principal da documentação gerada pelo Doxygen.